Neurocomputing 684 (2026) 133478

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Curiosity-driven decision causal-convolutional transformer with adaptive training for offline-to-online multi-agent reinforcement learning

Bohang Shi [a,1], Weiqiang Jin [a,b,c,1], Biao Zhao [a,*]

[a] School of Information and Communications Engineering, Faculty of Electronic and Information Engineering, Xi'an Jiaotong University (XJTU), Xi'an, 710049, Shaanxi, China

[b] Department of Electrical and Electronic Engineering, University of Hong Kong (HKU), Pokfulam, 999077, Hong Kong Special Administrative Region of China

[c] Department of Computer Science and Engineering (CSE), The Chinese University of Hong Kong (CUHK), Shatin, New Territories, 999077, Hong Kong Special Administrative Region of China

HIGHLIGHTS

- Proposing a curiosity-driven causal-convolutional MADT framework with adaptive optimization for offline-to-online MARL.
- Introducing a multi-agent causal-convolutional Transformer actor, MA-CCT, to enhance local dependencies in partially observable MDP.
- Designing a curiosity-driven exploration mechanism, CurExp to improve OOD generalization capabilities.
- Adopting a multi-stage adaptive learning-rate scheduler that adjusts optimization based on real-time performance.
- Conducting experiments on StarCraft-II SMAC, demonstrating superior performance over SOTA offline MARL methods.

ARTICLE INFO

Communicated by S. Zhang

Keywords:
Offline multi-agent reinforcement learning
Decision transformer
Local convolution filtering
Curiosity-driven exploration
Multi-stage adaptive learning

ABSTRACT

Offline multi-agent reinforcement learning (MARL) utilizes pre-collected datasets to learn policies that achieve near-optimal decision-making without interacting with the environment. While achieving encouraging results, current methods still face two major limitations: (1) suboptimal trajectories in the dataset provide imperfect guidance for policy learning, and (2) restricted data coverage results in weak generalization to out-of-distribution (OOD) states. To tackle these challenges, this paper introduces an integrated offline-to-online MARL framework named Curiosity-Driven Decision Causal-Convolutional Transformer with Adaptive Optimization (CDCT-AO), building upon and extending the Multi-Agent Decision Transformer (MADT). Specifically, CDCT-AO incorporates a new action sequence prediction module termed Multi-Agent Causal-Convolutional Transformer (MA-CCT), which integrates causal-convolution operators into a Transformer-based actor, a novel structure in offline MARL. MA-CCT is tailored to better capture localized temporal dependencies in partially observable multi-agent trajectories, thereby improving the policy's robustness when encountering unseen states. Furthermore, we propose an intrinsic curiosity-based exploration mechanism, CurExp, that uses prediction errors of the next state and action to stimulate agent exploration during fine-tuning. This mechanism effectively narrows the gap between offline data and actual environmental dynamics, promoting improved OOD generalization. Additionally, a Multi-Stage Adaptive Learning rate scheduler (MSAL) is introduced to dynamically adjust optimization parameters using online performance feedback. MSAL contributes to stable and efficient policy adaptation throughout the training process. To support reproducibility and future research, we have made the implementation of CDCT-AO publicly available on GitHub (Our code: https://github.com/albert-jin/CDCT-AO *[Accessed on 2025.08]*).

* Corresponding author.
Email addresses: Bh_567@stu.xjtu.edu.cn (B. Shi), weiqiangjin@stu.xjtu.edu.cn; jinwq4rl@hku.hk (W. Jin), biaozhao@xjtu.edu.cn (B. Zhao).

[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.neucom.2026.133478
Received 28 August 2025; Received in revised form 7 March 2026; Accepted 25 March 2026
Available online 31 March 2026
0925-2312/© 2026 Published by Elsevier B.V.

1. Introduction

Multi-Agent Reinforcement Learning (MARL) [1,2] is a crucial research direction for constructing multi-agent intelligent cooperation systems [3] and effectively addresses the challenges of decentralized partially observable Markov decision processes (Dec-POMDPs) [4], where multiple agents interact in environments with partial observations, leveraging intricate cooperation to complete practical complex tasks. In recent years, MARL has gradually garnered significant attention, achieving remarkable success across diverse practical domains, including smart housework automation [5,6], multi-robot production systems [7,8], and multi-robot disaster rescue [9,10].

To concretely illustrate the practical problem setting, in this paper, we focus on multi-agent coordination tasks in partially observable environments, where real-world interactions are costly, risky, or impractical. Examples include autonomous vehicle fleets navigating complex traffic scenarios, multi-robot disaster response, and smart factory automation. In these scenarios, offline trajectory datasets collected from prior demonstrations provide the initial guidance for policy learning, while limited online fine-tuning is performed to adapt to new environmental dynamics. This clarifies the physical meaning of the distribution shift challenge and highlights the necessity for agents to generalize effectively to out-of-distribution (OOD) states. [11,12]

However, conventional MARL algorithms predominantly rely on online environment interaction, where agents learn through trial-and-error by continuously collecting new experiences. While effective in simulated or controlled settings, this paradigm faces critical limitations in real-world applications where interaction is costly, risky, or impractical, such as disaster rescue [9,10], autonomous driving [11,13], and healthcare [14,15]. The above challenges existing in online MARL motivate the development of the offline MARL paradigm, which learns exclusively from static datasets without active environment interaction. Offline MARL represents a distinct supervised learning paradigm that leverages static offline datasets to model the Dec-POMDP characteristics embedded in expert demonstrations. By efficiently extracting policies that achieve performance comparable to the original expert trajectories, offline MARL significantly reduces the need for costly online environment interaction. This makes it an effective framework for developing sophisticated multi-agent intelligent systems.

To date, a substantial body of theoretical research has been conducted and diverse innovative offline MARL algorithms have been proposed, which can be roughly divided into two mainstream training paradigms. One is based on temporal difference learning (TD) [16–19], which follows the individual Q-value prediction-based actor-critic framework with additional policy constraints, represented by *batch-constrained Q-learning* (BCQ) [20], *conservative Q-learning* (CQL) [16,19], *implicit Q-learning* (IQL) [17], and *random ensemble mixture* (REM) based on uncertainty estimation [21]. Another paradigm is based on behaviour-imitation (or behaviour-clone) that utilizes trajectory auto-generative optimization [22–25], which mainly includes *Decision Transformer* (DT) [22], *Online Decision Transformers* (ODT) [26], *Decision Convformer* (DC) [23], and *Multi-Agent Decision Transformer* (MADT) [27]. Compared to TD-based methods, the imitation clone-based paradigm offers simpler modeling by directly cloning expert behaviors and benefits from the superior sequence modeling capacity of Transformer-like architectures, which makes trajectory imitation increasingly prominent in offline MARL.

Despite these achievements, the primary performance limitations and challenges lie in the divergence between the learned policy and the initial behavior policy that generated the dataset, i.e., distributional shift [26,27]. Specifically, the learned policy inevitably diverges from the behavior policy, which ultimately compromises value estimation reliability and optimal decision-making. Moreover, the inadequacy of offline datasets, which fail to comprehensively cover the state space encountered in real-world interactions, leads to poor generalization of the learned policy.

As shown in Fig. 1, the discrepancies between offline training and online environment interactions remain unresolved, resulting in the OOD problem. In such cases, agents may take sub-optimal actions when faced with unseen states beyond their learned policy. Thus, practical implementations typically require a fine-tuning phase [26,27]. The pre-trained policies on offline data are further refined through limited environmental interactions, as this online component helps mitigate distributional shift by allowing the policy to adapt to real state distributions [3,12].

Although behaviour imitation based methods, such as MADT [27], demonstrate superior capability in predicting the next action, the decision-making of MARL agents inherently exhibits a localized association pattern between adjacent timesteps due to the Markov property. This stands in contrast to traditional NLP tasks [28,29] and is critical for identifying the underlying transition dynamics and reward function

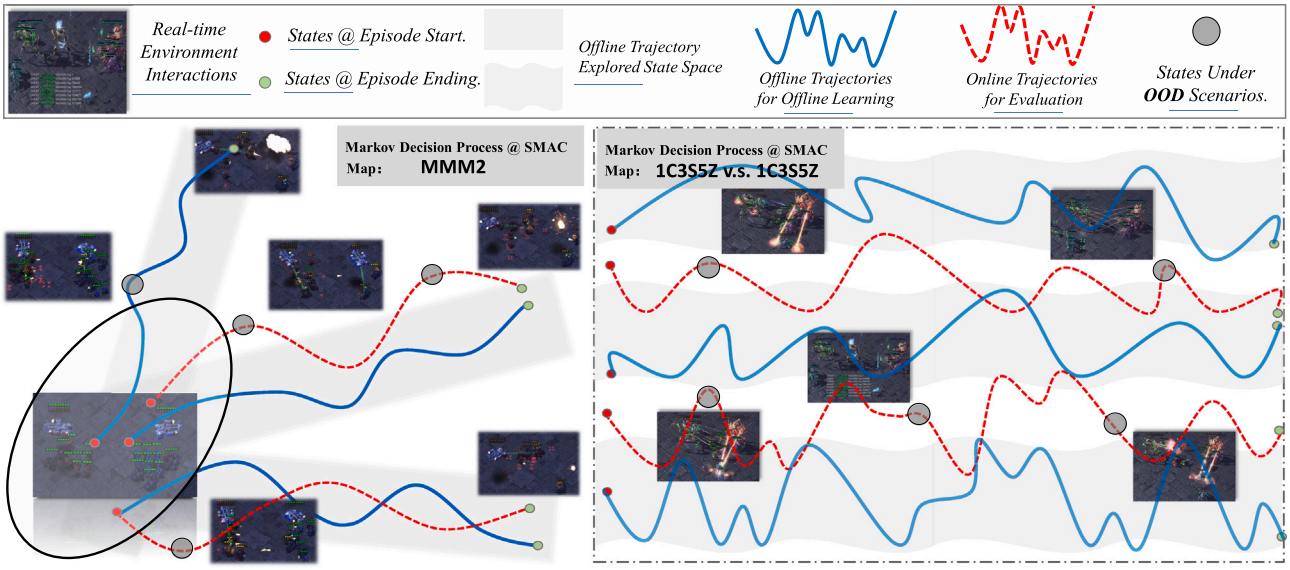


Fig. 1. The two typical SMAC scenarios on '`MMM2`' and '`1c2s5z vs.1c3s6z`' maps show the discrepancies between offline experience-based training and online evaluation. Both depicted multi-agent coordination processes begin from similar initial states, progress through diverse intermediate states (represented by the curves), and ultimately converge to goal states. Under such conditions, the practical interaction states may not be fully covered by offline data, leading to OOD states.

of an MDP. But existing approaches often fail to capture this unique temporal dependency structure in MDPs. Thus, there is a need to develop a novel actor architecture capable of modeling inter-modal relationships while extracting localized temporal patterns, thereby better adapting to the development of offline MARL-based intelligent systems. Meanwhile, since offline pre-trained policies often exhibit conservative exploration due to their reliance on fixed datasets, the fine-tuned policy during online rollout tends to inadequately adapt to OOD states. Thus, it is essential to design an exploration accelerating mechanism for online fine-tuning that incentivizes comprehensive state coverage beyond the offline dataset, thereby enhancing the overall robustness and intelligence of multi-agent decision-making systems. Given the significant performance fluctuations observed in multi-agent joint policies during offline-to-online training, the exploration diversity plays a pivotal role in policy optimization. To ensure both stability throughout the training process, it is necessary to develop an adaptive optimization module that dynamically adjusts learning capabilities based on agents' real-time feedback, thereby achieving superior training results.

Through the aforementioned challenges and issues, we propose a novel offline-to-online MARL framework, called ***C**uriosity-**D**riven **C**onvolutional Transformer with **A**daptive **O**ptimization* (**CDCT-AO**). To summarize, CDCT-AO comprises three primary innovations:

- From the network architecture perspective, we introduce a novel Transformer-like actor, **M**ulti-**A**gent **C**ausal-**C**onvolutional **T**ransFormer (MA-CCT), for better generalization and fitting of offline expert trajectories;
- During the online fine-tuning stage, we incorporate a novel **Cur**iosity-driven **Exp**loration mechanism (**CurExp**), to facilitate comprehensive online state coverage and avoid OOD scenarios;
- We also adopt a **M**ulti-**S**tage **A**daptive **L**earning-rate scheduler (**MSAL**) to enable dynamic parameter adjustment based on real-time performance feedback.

In specific, the MA-CCT actor in CDCT-AO is designed to overcome the limitation prevalent in offline MARL, which fails to effectively capture localized temporal patterns between adjacent timesteps. MA-CCT designs a novel causal-convolution based token-mixer to replace the multi-head self-attention (MHSA) in Transformer-based action predictors. This adaptation proves particularly suitable for capturing the local-association structures inherent in MDP data sequences, thereby more effectively modeling the Markovian properties essential for MARL. Recent architectural innovations for sequence-based decision models often focus on enhancing local temporal pattern capture. DC [23] replaces self-attention with a spatial convolution mixer, while LSDT [24] employs a dual-branch architecture with separate convolution and attention pathways. While StARformer [30] incorporates a Markovian bias by first modeling state-action-reward triples in a local window. In contrast, our proposed MA-CCT fundamentally diverges by employing a pure causal-convolution operator as the core token mixer within a unified Transformer-style block. This design enforces a strict causal information flow aligned with MDPs, and differs from the parallel designs of prior works. To the best of our knowledge, this represents the first application of causal-convolution in offline MARL settings.

To address the critical challenge of limited state coverage during offline-to-online fine-tuning, we further propose CurExp. CurExp is an intrinsic motivation mechanism designed to address the OOD generalization challenge caused by limited offline data coverage. CurExp employs a forward dynamics model to predict next-state transitions, using the resulting prediction error to guide agents toward uncertain yet meaningful states. Experiments show CurExp effectively bridges the offline-online distribution gap, improving OOD robustness in unseen scenarios, making it particularly valuable for real-world complex applications.

Finally, we designed a multi-stage adaptive learning rate scheduler, MSAL, to enhance both the offline training and online fine-tuning for agents. MSAL employs a *warmup phase* to initialize the learning rate gradually, preventing early instability. Then, an *exploration phase* is initiated, where learning rate are boosted to accelerate policy adaptation in the new environment. When performance reaches a high threshold, it enters a *fine-tuning phase*, reducing the learning rate for stable convergence. MSAL increases the learning rate to escape poor performance and decreases the learning rate to refine strong policies in a self-adaptive fashion, improving optimization robustness.

Through extensive experiments on the *StarCraft-II Multi-Agent Challenge* (SMAC) benchmark, we validate the effectiveness and superiority of our proposed CDCT-AO framework for offline MARL. The results on SMAC demonstrate that our method significantly outperforms the latest offline MARL approaches, including TD-based algorithms like *multi-agent BCQ* (BCQ-MA) [20], *multi-agent IQL* (IQL-MA) [17], *multi-agent CQL* (CQL-MA) [16], and behaviour imitation based algorithms like *Behavior Cloning* (BC) [31] and MADT [22,27], in terms of both convergence efficiency and ultimate win-rate performance.

Overall, CDCT-AO presents significant implications for offline-to-online MARL research, particularly in addressing challenges such as OOD generalization, exploration efficiency, and adaptive optimization. By addressing these challenges, CDCT-AO enables more practical real-world intelligent systems by solving limitations in data efficiency, safety, and adaptability. We hope this inspires further innovations in hybrid architectures, intrinsic exploration mechanisms, and adaptive learning strategies for developing next-generation MARL systems. Our contributions in this work can be summarized as follows:

- In this work, we identify two key challenges in offline MARL: imperfect supervision from suboptimal policies and poor OOD generalization due to limited data coverage. To address these, we propose the *Curiosity-Driven Decision Causal-Convolutional Transformer with Adaptive Optimization* (CDCT-AO) framework;
- CDCT-AO introduces three specialized innovative modules: 1. MA-CCT, a novel Transformer-based actor integrated with causal-convolution filters for trajectory modeling, 2. CurExp, an intrinsic curiosity exploration mechanism that utilizes the MDP transition prediction error for sufficient exploration, and 3. MSAL, a dynamic optimization scheduler that automatically adjusts optimization dynamics through multiple distinct phases for adaptive learning.
- Through extensive experiments on SMAC, we demonstrate that CDCT-AO significantly outperforms state-of-the-art offline MARL methods in convergence, average return, and win-rate, including TD-based and behaviour imitation-based approaches.
- We provide our code[2] publicly to facilitate reproducibility and encourage further research in offline-to-online MARL.

The remainder of this paper is organized as follows. In Section 2, we review related work in offline MARL and Transformer-based decision models. Section 3 presents the formal problem formulation on online/offline MARL and Dec-POMDPs. In Section 4, we detail the CDCT-AO framework including its three core components: MA-CCT actor, CurExp mechanism, and MSAL strategy. Section 5.1 presents the experimental settings, including the adopted multi-agent SMAC environment, baselines, evaluation metrics, and hyper-parameters. Section 5.2 discusses experimental results comparing CDCT-AO with state-of-the-art (SOTA) methods. Section 6 presents several representative case studies to enhance the readers' understanding of OOD generalization and multi-agent exploration behaviors. Section 5.3 presents a comprehensive ablation study to validate the efficacy of each proposed module in our CDCT-AO. Finally, Section 8 concludes the paper and outlines potential improvements and future directions for offline-to-online MARL research.

[2] Our code for CDCT-AO: https://github.com/albert-jin/CDCT-AO. *[Accessed on 2025.08]*

2. Related work

2.1. Traditional online MARL

Online MARL, as an extension of online RL in a single-agent context, has demonstrated remarkable success in modeling complex multi-agent collaboration for diverse real-world applications. Online MARL methods can be broadly classified into three categories based on the agents' access to global environmental information during the training and execution phases: *Centralized Training with Decentralized Execution* (CTDE), *Decentralized Training with Decentralized Execution* (DTDE) and *Centralized Training with Centralized Execution* (CTCE) [3].

Among these, CTDE is a widely adopted paradigm in MARL, enabling collaborative training with decentralized execution. MAPPO extends PPO to multi-agent systems using centralized value functions [32]. MADDPG combines deep learning and deterministic policy gradients for continuous action spaces [33]. VDN decomposes the value function into individual agent value functions [34]. COMA leverages counterfactual reasoning to compute individual agent policies [35]. QMIX uses a mixing network to combine individual value functions into a team value function [36]. Weighted QMIX adapts to heterogeneous environments by assigning importance-based weights [37]. MA-MIX introduces a multi-head attention mechanism to enhance cooperative MARL [38]. QTRAN improves coordination through counterfactual advantage functions [39]. Qatten focuses on improving coordination by addressing training inefficiencies [40]. QPLEX factorizes Q-values to capture complex agent interactions [41]. MATRPO extends TRPO to multi-agent systems with trust-region constraints [42]. LICA enables implicit coordination [43]. RODE uses recurrent networks for decentralized evaluation [44].

However, traditional MARL heavily relies on costly and risky online interaction, which brings significant costs and safety problems [7,9,10]. For example, in autonomous driving [13] or emergency response systems [10], this online training approach faces challenges such as high computational costs, and safety risks, potentially leading to collisions, or mission delays [12,45].

2.2. Offline MARL

To mitigate these limitations while preserving learning effectiveness, offline MARL has emerged as a promising alternative that learns multi-agent cooperative policies from pre-collected offline datasets, thereby avoiding risky online interactions while maintaining policy superiority [9–11,13]. Theoretically, it shifts the learning process from direct environment interaction to batch dataset utilization. Offline MARL, equipped with powerful function approximation, can transform datasets into highly capable multi-agent intelligent decision-making systems. This means that any multi-agent system developers and researchers with sufficient data could potentially convert it into an optimized policy tailored to their specific objectives [12,45].

Mainstream offline MARL approaches primarily follow two paradigms: (1) Temporal-difference (TD) learning with conservative policy constraints, e.g., BCQ [20], CQL [16,19], and (2) Behaviour imitation leveraging sequence modeling, e.g., DT [22], StARformer [30], MADT [27], Long-Short Decision Transformer (LSDT) [24].

2.2.1. TD-based offline MARL

For TD-based offline training, its standard process still retains the actor-critic framework, where agents' policies are optimized by leveraging the signals of rewards and cumulative returns from expert trajectories. Based on this, these methods jointly optimize both the critic's Q-value estimation and the actors' policy gradients to iteratively improve multi-agent policies [1,12].

BCQ-MA [20] uses a state-conditioned generative model to restrict the policy, ensuring actions remain close to the offline dataset to prevent Q-value overestimation due to distributional shift. ICQ-MA [18] effectively mitigates extrapolation error in offline MARL by only trusting state-action pairs present in the dataset for value estimation, avoiding unseen pairs through a SARSA-like approach and decomposing the joint-policy under an implicit constraint. CQL-MA [16] mitigates value overestimation by learning a conservative Q-function that lower-bounds the true policy value, preventing errors from distributional shift while maintaining policy improvement guarantees. *Counterfactual CQL* (CFCQL) [19], a multi-agent extension of CQL, addresses overestimation in offline MARL by applying per-agent conservative regularization in a counterfactual manner, rather than treating all agents as a single high-dimensional entity.

While TD-based approaches have made significant progress, they still suffer from several crucial limitations: (1) The reliance on value prediction optimization-based methods such as conservative Q-learning [16,19] may lead to value overestimation, resulting in the learned policies being overly conservative. (2) These approaches suffer from inherent deficiencies in handling long-term dependencies and complex temporal relationships between actions and rewards, such as long-horizon planning [12,45].

2.2.2. Behaviour imitation-based offline MARL

For imitation-based offline training, the goal is to learn policies by directly mimicking expert behavior from pre-collected trajectories. By leveraging prior offline trajectory sequences of states, actions, and rewards in the offline data, these methods replicate the decision-making process of experts, refining the policy network to better predict optimal next actions.

DT [22] is a representative sequence modeling-based offline RL approach that leverages the powerful generalization capability of Transformer architectures to reformulate reinforcement learning as conditional trajectory generation. *Q-learning Decision Transformer* (QDT) [46] is an enhanced DT framework that solves the problems of inadequate trajectory stitching capability and sensitivity to return-to-go specification of DT. QDT synergizes dynamic programming with sequence modeling through intelligent trajectory relabeling by leveraging value function estimates to dynamically adjust return-to-go targets. DC [23] replaces DT's conventional multi-head attention module with one-dimensional convolution filters to better capture local Markovian dependencies in offline RL datasets. LSDT [24] combines parallel self-attention and dynamic convolution branches to simultaneously capture global long-range dependencies and fine-grained local Markovian patterns, enabling robust performance across diverse offline datasets. MADT [27], a multi-agent extension of DT [22], employs a transformer-based architecture that formulates MARL as a sequence modeling problem, enabling offline pre-training and efficient transfer to diverse downstream tasks with strong few-shot and zero-shot generalization capabilities. *Online Decision Transformers* (ODT) [26] introduces a novel sequence-level entropy regularizer within the autoregressive modeling objective to blend offline pretraining with online finetuning in a unified fashion. *Offline MARL with Knowledge Distillation* (OMARL-KD) [47] adopts Transformers and trains a centralized teacher policy to identify optimal behaviors from offline data and distills both policy features and inter-agent relationships into decentralized student policies.

Compared with TD-based value optimization approaches, behaviour imitation based methods exhibit several distinct advantages, including: simplified modeling by directly leveraging supervised learning on demonstration trajectories, superior temporal modeling capability through Transformer-like architectures, and avoiding the complex computation and compounding errors inherent in iterative Bellman updates.

2.3. Architectural and algorithmic distinctions for various transformer actors in offline MARL

Here, we conduct a systematic comparison of related Transformer-like actor's architectures [12,45].

Firstly, Decision ConvFormer (DC) [23] replaces the Transformer's self-attention with a spatial convolution mixer, focusing on local pattern extraction. Similarly, Long-Short Decision Transformer (LSDT) [24] employs a dual-branch architecture, using a dedicated convolution branch in parallel to a self-attention branch to capture both local and global dependencies. Several recent works have also explored reinforcement learning paradigms that leverage offline or limited interaction data to enhance learning performance. For example, using offline trajectories to improve RL sample efficiency has been investigated in procedurally generated environments, showing the positive impact of trajectory diversity and offline pre-training on generalization performance [48]. Additionally, information-directed exploration strategies in reinforcement learning have been proposed for autonomous search and estimation problems [49], demonstrating the role of exploration in practical multi-agent systems. Reinforcement learning has also been studied for optimal interaction intensity in cooperative multi-agent tasks, further illustrating the broad relevance of RL techniques in complex decision-making problems [50]. In contrast, our proposed MA-CCT uniquely integrates a causal-convolution operator as the core token mixer within a single Transformer-style block. This design ensures that each token's representation is synthesized exclusively from its preceding context, creating a strict causal information flow that is more aligned with the forward temporal nature of MDPs. Furthermore, while StARformer [30] introduces a Markovian inductive bias by first modeling short-term "state-action-reward" representations within a local window, it relies on standard self-attention for these local and subsequent global modeling stages. MA-CCT fundamentally differs by replacing the global self-attention mechanism altogether, providing a consistent, localized mixing operation across the entire network depth. Overall, MA-CCT shares the high-level motivation with several recent architectures [19,25,51] that aim to inject an inductive bias for local temporal dependencies into sequence-based decision models.

To the best of our knowledge, our proposed MA-CCT represents the first application and systematic exploration of a purely causal-convolutional architecture specifically within the offline MARL domain. This introduces the additional challenge of learning coordinated policies from decentralized, partially observable trajectory data, a setting not addressed by the aforementioned single-agent focused models.

3. Preliminary

3.1. Dec-POMDP and CTDE-based MARL

Definition of Dec-POMDP. Formally, conventional online MARL approaches operate within the Dec-POMDP paradigm. The Dec-POMDP is formally defined by $\langle N, S, \{A_i\}_{i\in I}, P, \{R_i\}_{i\in I}, \{O_i\}_{i\in N}, \mathcal{O}, \gamma\rangle$, where N is a finite set of n agents, S represents the global state space with true state $s \in S$, A is the action space where each agent $i \in N$ selects $u_i \in A$ forming a joint action $\mathbf{u} = [u_i]_{i=1}^n \in A^n$, $P(s' \mid s, \mathbf{u}) : S \times A^n \times S \to [0,1]$ is the state transition function, and the shared reward function is $R(s,\mathbf{u}) : S\times A^n \to \mathbb{R}$. The observation space is denoted by $\mathcal{O}$, and $O(s,i) : S \times N \to \mathcal{O}$ generates the local observation $o_i \in \mathcal{O}$ for each agent i. The discount factor is denoted as $\gamma \in [0,1)$. During a standard Dec-POMDP process, at each timestep t, agent i receives an observation o_i^t from $O(s^t, i)$ based on the global state s^t, maintaining an action-observation history $\tau_i \in \mathcal{T} = (\mathcal{O}\times A)^* \times \mathcal{O}$. Using a stochastic policy $\pi_i(u_i^t \mid \tau_i, u_i^{t-1})$ the agent selects its individual action u_i^t. All agents' individual actions then form the joint action $\mathbf{u} = [u_i^t]_{i=1}^n$, which is executed in the environment. The environment responds to $\mathbf{u}^t$ by providing: the immediate reward $r^t = R(s^t, \mathbf{u}^t)$; transitioning to the next state $s^{t+1} \sim P(\cdot \mid s^t, \mathbf{u}^t)$; and generating new observations $o_i^{t+1} = O(s^{t+1}, i)$ for each agent i.

Definition of CTDE-based MARL. In the training phase of mainstream MARL under CTDE, each agent's decentralized actor optimizes its individual policy using only its local observations o_i and action-observation history τ_i, while the centralized critic utilized during training has access to the global state s to facilitate coordinated learning. After training, agents execute their learned policies based on their local observations o_i and individual trajectory history τ_i, without needing access to the global state s or the supervision of the centralized critic. The optimization process of CTDE-based MARL is as follows:

Specifically, at each timestep t, each agent i first receives individual observations o_i^t from the global state s^t of the environment, where $o_i^t = \mathcal{O}_i(s^t)$ and $\mathcal{O}_i$ is the observation function for agent i, which contains partial information about the global state s^t. Given its local action-observation history τ_i^t which includes its individual observations o_i^t at timestep t, each agent i uses its respective decentralized policy $\pi_i(u_i^t \mid \tau_i^t)$ to determine the next action u_i^t.

Subsequently, all agents' actions are used to construct the joint action $\mathbf{u^t} = [u_i^t]_{i=1}^N \in A^N$. $\mathbf{u^t}$ is used to compute the joint action-value $Q^{\text{tot}}(s, \mathbf{u})$ by adopting the centralized critic function $Q^{\text{tot}}(\cdot)$ with access to the global state s and joint history $\boldsymbol{\tau} = (\tau_1, \dots, \tau_N)$.

Critically, in value-based CTDE methods, the joint action-value function $Q^{\text{tot}}(\cdot)$ is decomposed from individual agent utilities $Q_i(\tau_i, u_i; \phi_i)$ through a mixing mechanism that respects the *Individual-Global Maximization (IGM)* principle [34,39]. Each agent maintains its own Q-network with parameters ϕ_i that estimates its individual action Q-value Q_i based solely on its trajectory history τ_i.

The centralized joint action-value function is constructed as:

$$Q^{\text{tot}}(\boldsymbol{\tau}, \mathbf{u}, s; \theta) = f_{\text{mix}}(Q_1(\tau_1, u_1; \phi_1), \dots, Q_N(\tau_N, u_N; \phi_N) | s; \theta_{\text{mix}}) \tag{1}$$

where f_{mix} is a mixing network parameterized by θ_{mix}, and $\theta = \{\phi_1, \dots, \phi_N, \theta_{\text{mix}}\}$. To ensure consistency between centralized training and decentralized execution, the decomposition must satisfy the IGM condition:

$$\arg\max_{\mathbf{u}} Q^{\text{tot}}(\boldsymbol{\tau}, \mathbf{u}, s) = (\arg\max_{u_1} Q_1(\tau_1, u_1), \dots, \arg\max_{u_N} Q_N(\tau_N, u_N)) \tag{2}$$

This guarantees that maximizing the centralized Q^{tot} is equivalent to each agent independently maximizing its own Q_i, enabling optimal decentralized execution under partial observability.

The joint action-value function follows the Bellman optimality equation adapted for partially observable multi-agent settings:

$$Q^{\text{tot}}(\boldsymbol{\tau}, \mathbf{u}, s) = \mathbb{E}_{s', \mathbf{o}'}\left[r(s, \mathbf{u}) + \gamma \max_{\mathbf{u}'} Q^{\text{tot}}(\boldsymbol{\tau}', \mathbf{u}', s') | \boldsymbol{\tau}, \mathbf{u}, s \right] \tag{3}$$

where $r(s, \mathbf{u})$ is the immediate reward, γ is the discount factor, $\mathbf{o}' = (o_1', \dots, o_N')$ are next-step local observations, and $\boldsymbol{\tau}'$ denotes the updated joint histories.

The above formulations explicitly account for several key challenges that distinguish MARL from single-agent RL: (1) *Partial observability*, where each Q_i depends only on τ_i rather than s; (2) *Multi-agent credit assignment*, addressed through f_{mix} which learns to apportion global reward to individual agents; and (3) *Non-stationarity during training*, mitigated by CTDE where the centralized Q^{tot} provides a stable learning signal despite the changing policies of other agents.

During training, the joint Q-value function is approximated using the neural networks of both actors and the centralized critic, parameterized by θ, as $Q^{\text{tot}}(\boldsymbol{\tau}, \mathbf{u}; \theta)$, where $\boldsymbol{\tau} = (\tau_1, \dots, \tau_N) \in \mathcal{T}^N$. The training minimizes the TD-error using the loss function:

$$\mathcal{L}(\theta) = \mathbb{E}_{(\boldsymbol{\tau}, \mathbf{u}, r, \boldsymbol{\tau}') \sim \mathcal{D}}\left[\left(r + \gamma V^{\text{tot}}(\boldsymbol{\tau}'; \theta^-) - Q^{\text{tot}}(\boldsymbol{\tau}, \mathbf{u}; \theta) \right)^2 \right], \tag{4}$$

where $\mathcal{D}$ is the replay buffer, and θ^- represents the parameters of the target network. Each agent's decentralized policy π_i is derived from the individual Q-value $Q_i(\tau_i, u_i; \phi_i)$ that factorizes into the joint Q^{tot} through mixing networks or other coordination mechanisms.

3.2. Offline MARL

In practical multi-agent systems, acquiring real-time environmental interactions is often expensive or unsafe. Offline MARL addresses this

by learning from pre-collected trajectory datasets, which encode the experiences of multiple agents under prior policies. These datasets serve as a proxy for real-world interactions, enabling policy learning without additional risk. However, because the datasets may not cover all relevant states, policies trained purely offline may perform poorly when encountering previously unseen OOD states. To mitigate this, limited online interactions are used during a fine-tuning phase, allowing agents to adapt and generalize safely to new scenarios.

Offline MARL learns agents' joint policies from a fixed dataset of previously collected sub-optimal experiences, rather than interacting with the environment in real-time. This learning paradigm is particularly useful in situations where real-time interactions with the environment are expensive, unsafe, or impractical.

In offline MARL, agents do not interact with the environment and instead rely on an offline dataset D that contains state transitions, actions, rewards, and next states observed in prior episodes. This dataset serves as the basis for training the agents and learning their policies without any further real-time exploration. Consider a intelligent system of N agents, each interacting within a shared environment. Given the offline dataset $D = \left\{\tau^{(k)}\right\}_{k=1}^{K}$, where K is the total number of trajectories, and each trajectory $\tau^{(k)}$ is defined as $\tau^{(k)} = (s_0, a_0, r_0, o_1, \ldots, s_T, a_T, r_T)$, where s_t is the state of the environment at time t, $a_t = (a_{t,1}, \ldots, a_{t,N})$ represents the joint action taken by all N agents at time t, $r_t = (r_{t,1}, \ldots, r_{t,N})$ is the reward vector received by the agents at time t, $o_t = (o_{t,1}, \ldots, o_{t,N})$ represents the joint observation of all agents at time t. The dataset is a collection of trajectories, where each trajectory represents the experience of the agents, gathered through the execution of joint behavior policies. This enables the system to estimate how each agent should act in future decision-making scenarios.

The objective in offline MARL is to learn the optimal decentralized policies $\pi^* = \left\{\pi_1^*, \ldots, \pi_N^*\right\}$ for each agent, where each π_i^* is learned from the offline dataset D, without interaction with the environment during the learning process, such that π^* maximizes the expected cumulative reward. The expected return $R(\pi)$ for a given joint policy π is defined as: $R(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \mathbf{a}_t)\right]$, where $\mathbb{E}_\pi$ denotes the expectation over trajectories generated by following policy π, s_t is the global state and $\mathbf{a}_t$ is the joint action at time t. The agents aim to find the optimal policy π^* that maximizes $R(\pi)$, using only the offline dataset D consisting of state-action-reward-next state tuples $(s_t, \mathbf{a}_t, r_t, s_{t+1})$ and without further environment interaction.

3.3. DT and its multi-agent variant, MADT

3.3.1. Decision transformer (DT)

DT [22] reformulates RL as a sequence modeling problem. It utilizes the Transformer [29,52] to model trajectories, treating them as sequences of *Return-to-Go* (`RTG`), `states`, and `actions`. At each timestep t, DT constructs a sub-trajectory of length K as its input:

$$\tau_{t-K+1:t} = (\hat{R}_{t-K+1}, s_{t-K+1}, a_{t-K+1}, \ldots, \hat{R}_{t-1}, s_{t-1}, a_{t-1}, \hat{R}_t, s_t), \tag{5}$$

where the Return-to-Go (`RTG`) $\hat{R}_t$ is the sum of all future rewards from timestep t, i.e., $\hat{R}_t = \sum_{t'=t}^{T} r_{t'}$.

In datasets containing numerous suboptimal trajectories, the label of `RTG` serves as a crucial indicator for distinguishing optimal behavior. DT learns to predict the action a_t at timestep t autoregressively, using either a cross-entropy or mean squared error (MSE) loss, thereby generalizing an optimal policy.

3.3.2. Multi-agent decision transformer (MADT)

MADT [27] extends DT to MARL. It leverages the Transformer's sequence modeling capabilities to learn from offline datasets and can be fine-tuned for online tasks. Next, we formulate MADT in terms of input, output and training target function.

Input. For a multi-agent setting with N agents, the input to the MADT at a single timestep t is a tuple containing the global state and all agents' local observations and actions. The input sequence for a context window of length C is constructed as: $\mathbf{X}_{1:C} = (\mathrm{h}_1, \mathrm{h}_2, \ldots, \mathrm{h}_C)$, where the token h_t at each timestep t is defined as: $\mathrm{h}_t = (s_t, o_t^1, a_t^1, o_t^2, a_t^2, \ldots, o_t^N, a_t^N)$. Here, s_t is the global state, o_t^i is the local observation of agent i, and a_t^i is the action taken by agent i at timestep t. The entire sequence $\mathbf{X}_{1:C}$ is embedded into a continuous vector space and combined with a positional encoding before being fed into the Transformer encoder.

Output. Then, MADT operates autoregressively to output target predictions. For each timestep t within the context window, the model consumes the sequence $\mathrm{h}_{1:t}$ and outputs a hidden representation $\mathbf{z}_t$ for that timestep: $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_C = \mathrm{Transformer}(\mathrm{h}_1, \mathrm{h}_2, \ldots, \mathrm{h}_C)$, where the hidden representation $\mathbf{z}_t$ is subsequently used by a linear output layer to predict the action distribution for each agent i at the *next* timestep, $t+1$. The predicted action $\hat{a}_{t+1}^i$ for agent i is:

$$\hat{a}_{t+1}^i = \arg\max\ p(a_{t+1}^i | \mathbf{z}_t; \theta) = \arg\max\ \mathrm{Linear}^i(\mathbf{z}_t)$$

where Linear^i is the output projection layer specific to agent i, and $p(X|Y)$ denotes the posterior probability of X given Y.

Training. The model is trained to predict the action a_t^i for each agent i at timestep t conditioned on the preceding trajectory $\mathrm{h}_{1:t-1}$. The training objective is to minimize the cross-entropy loss over the action space, while masking illegal actions via the available action mask avail_t^i:

$$\mathcal{L}(\theta) = -\frac{1}{C}\sum_{t=1}^{C}\sum_{i=1}^{N} \log p(a_t^i | \mathrm{h}_{1:t-1}; \theta) \cdot \mathbf{1}[\mathrm{avail}_{t-1}^i(a_t^i) = \mathrm{True}]$$

where C is the context length and θ represents the parameters of MADT.

After offline pre-training, the MADT model can be fine-tuned online using standard MARL algorithms (e.g., PPO). The pre-trained actor (policy) and critic (value) networks are loaded and updated with experience collected from the online environment, significantly improving sample efficiency and enabling strong few-shot and zero-shot generalization to new tasks.

4. Curiosity-driven decision causal-convolutional transformer with adaptive training

CDCT-AO is a hybrid offline-to-online approach that integrates multiple innovative modules to enhance pure offline MARL methods. Here, we provide an overview of the overall model architecture, which comprises three core components, MA-CCT (Section 4.1), CurExp (Section 4.2), and MSAL (Section 4.3), addressing critical challenges such as OOD issues and discrepancies between offline training and real-world deployment.

As illustrated in Fig. 2, the illustration of CDCT-AO is organized into two clearly distinguished and visually separated panels:

- (1) [Left] The widely adopted pre-training and fine-tuning paradigm in natural language processing (NLP), which is included solely as a conceptual analogy rather than a component of our method. This analogy is used to help us intuitively understand the motivation behind our offline-to-online MARL design, particularly the role of offline pre-training followed by online adaptation[3];
- (2) [Right] The core method illustration of our CDCT-AO Framework, which consists of two training phases: Offline Training and Practical Online Fine-tuning, following a paradigm similar to that of NLP fine-tuning scenarios.

Specifically, on the left side (*i.e.,* part.A) of Fig. 2, it depicts the pre-training and fine-tuning stages of the model, focusing on NLP tasks [28,29]. The pre-training phase shown in left-top involves learning from a large, unlabeled corpus through unsupervised tasks like Masked Language Modeling (MLM), where a pre-trained model is optimized

[3] Importantly, this NLP pipeline is not part of the CDCT-AO algorithm, and no NLP components are used in the proposed method. Its role is purely illustrative and motivational.

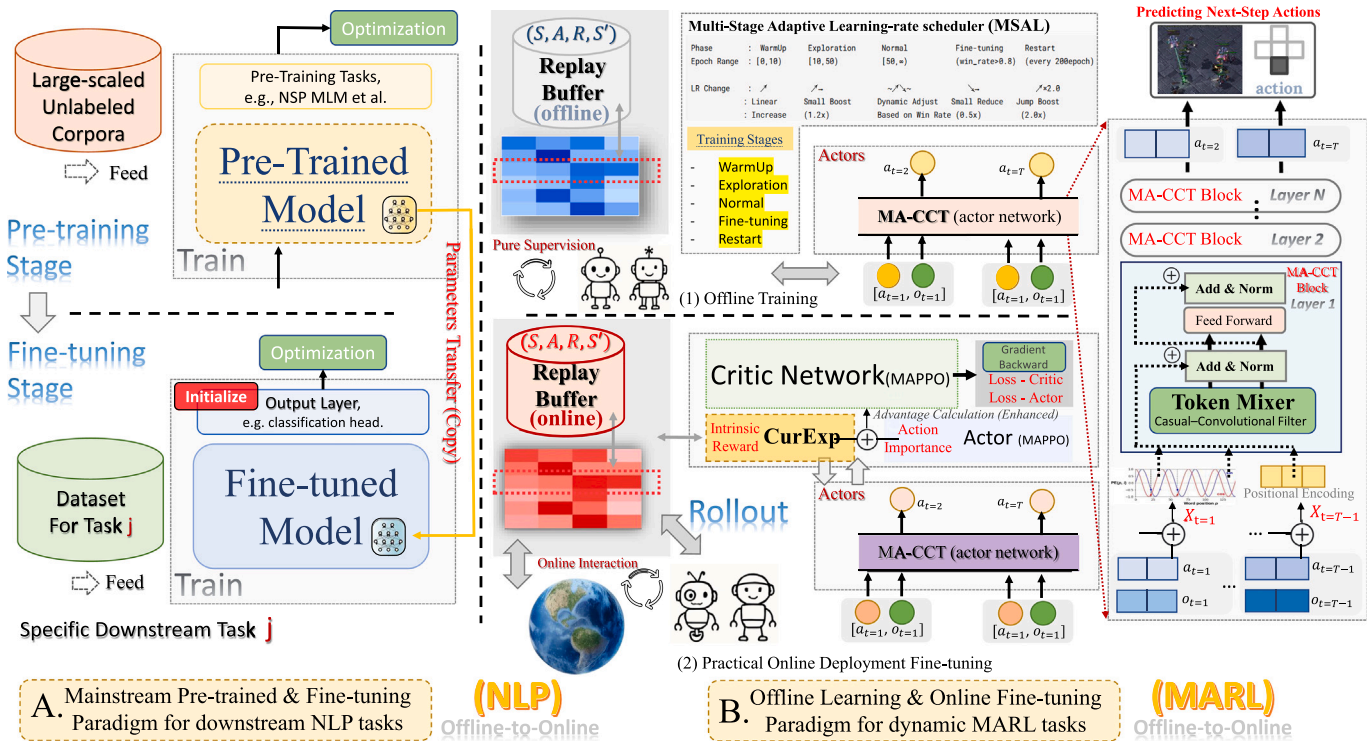


Fig. 2. The overall architecture Overview of *Curiosity-Driven Decision causal-convolutional Transformer with adaptive optimization* (CDCT-AO) for Offline-to-Online MARL. Part a (Left) illustrates the NLP pre-training and fine-tuning paradigm as a conceptual analogy only, introduced to motivate the offline-to-online learning strategy; part B (Right) presents the proposed CDCT-AO framework, which constitutes the actual method of this work and includes three core modules: MA-CCT, CurExp, and MSAL.

on a dataset tailored for a specific downstream task. As shown in left-bottom, during the fine-tuning phase, the pre-trained model undergoes further optimization on task-specific datasets to adapt to the new task. Meanwhile, on the right side (*i.e.,* part.B) of Fig. 2, the offline-to-online MARL of our CDCT-AO includes the (1) *Offline Training* and (2) *Practical Online Fine-tuning* phases. In the offline training phase, expert demonstrations are used to pre-train the model without real-time interaction with the environment. The MA-CCT actor, a crucial component of CDCT-AO, is used for capturing localized temporal dependencies between actions and observations. By integrating causal-convolution filters, MA-CCT improves the model ability to handle long-horizon planning and complex decision-making tasks in multi-agent systems. However, to encourage online exploration of more OOD states, CurExp is employed during online fine-tuning. CurExp encourages exploration of uncertain states to bridge the distributional gap and promote effective online adaptation. Furthermore, MSAL is adopted during both offline and online stages, which dynamically adjusts the learning rates during training to ensure robust optimization and prevent overfitting or slow convergence.

4.1. Multi-agent causal-convolutional TransFormer (MA-CCT)

As depicted in Fig. 3, the figure illustrates the architecture of the *Multi-Agent Causal-Convolutional Transformer* (MA-CCT) module, with three distinct sections: (1) Basic Transformer Actor; (2) Multi-Head Self Attention; and (3) Causal-Convolutional Filter.

Basic Transformer Actor. The bottom of Fig. 3 shows a basic transformer architecture used as an actor for decision-making. This structure typically includes the components of *initial token mixer module, add & normalization layers*, and *feed-forward layers*. This network begins with the input tokens $\mathbf{I}_t$, $t \in [0, T)$, which represent a sequence of actions or states over time. The attention mechanism processes these inputs to understand relationships between them and makes predictions about the next action a_{t+1} in a sequence, based on historical data.

Token Mixer of MHSA. As shown in the top-left of Fig. 3, the MHSA mechanism is at the heart of the Transformer architecture [29]. It allows the model to focus on different parts of the input sequence to understand the relationships between tokens (e.g., words in NLP or actions and states in RL) at different time steps.

For each input sequence, the model learns $Query(Q)$, $Key(K)$, and $Value(V)$, which are computed by multiplying the input sequence $\mathbf{I}$ by learned weight matrices: $Q = I\ W_Q$, $K = I\ W_K$, $V = I\ W_V$, where W_Q, W_K, and W_V are learned parameter matrices of appropriate dimensions.

The attention scores, which measure how much focus each token should have on others, are computed using the scaled dot-product between the Query (Q) and Key (K) matrices, followed by the *SoftMax* function to normalize the scores:

$$\text{Attention}(Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{6}$$

where Q is the Query matrix, d_k is the dimensionality of the keys, K^T is the transpose of the Key matrix, and $SoftMax(\cdot)$ ensures that the attention scores sum to 1.

The *Multi-Head* mechanism extends this process by employing multiple sets of these learned weight matrices in parallel. Each head computes its own independent attention output, allowing the model to capture different types of relationships and dependencies within the sequence. The outputs from all heads are then concatenated into a single matrix and projected using a final learned weight matrix W^O to produce the final multi-head attention output.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{7}$$

where each individual head is computed as $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$. This concatenated and projected output

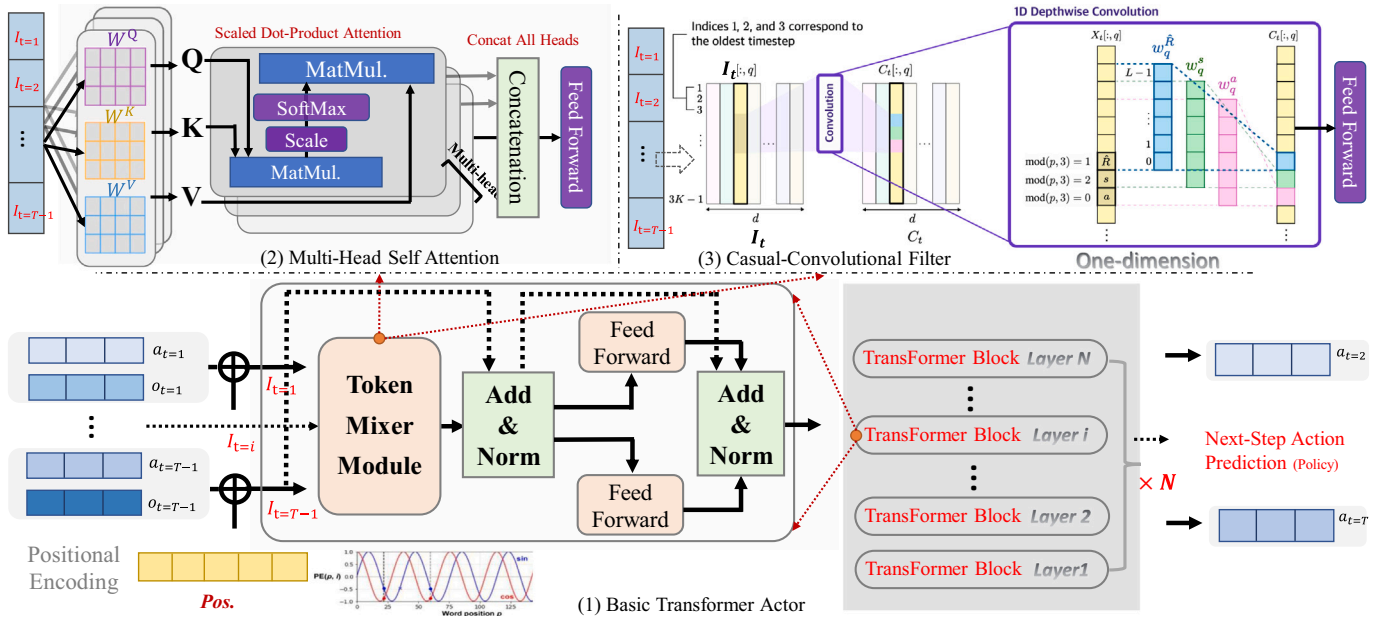


Fig. 3. Architecture of Multi-Agent Causal-Convolutional Transformer (MA-CCT). The three core architectural views of the MA-CCT module are illustrated as: **(1) Basic Transformer Actor** (bottom): Shows the overall sequence modeling pipeline where input tokens $\mathbf{I}_t$ (state, action, RTG) are processed through iterative token mixing and feed-forward layers to predict the next action a_{t+1}. **(2) Multi-Head Self-Attention (MHSA)** (top-left): Details the standard Transformer token mixer, where queries (Q), keys (K), and values (V) compute attention scores via scaled dot-product to capture global dependencies across all timesteps. **(3) Causal-Convolutional Filter** (top-right): Our proposed replacement for MHSA. It employs separate 1D depthwise convolutions for `State`, `Action`, and `RTG` token streams to capture localized, temporal (Markovian) transitions. A subsequent shared MLP (within the MetaFormer block) fuses these modality-specific features to enable cross-modal reasoning (e.g., action-to-state influence).

is then passed through the subsequent feed-forward layers and residual connections within the transformer block.

Casual-Convolutional Token Mixer of our MA-CCT. As shown in the top-right of Fig. 3, MA-CCT replaces the multi-head self-attention mechanism from the token-mixer in traditional Transformers with a *Causal-Convolutional Filter*. This filter processes tokens, i.e., the embeddings for `State`, `Action`, `RTG`, across different time steps. By using 1D depthwise convolutions, MA-CCT captures the temporal relationships between neighboring tokens.

Specifically, for each time step t, the input sequence I_t consists of states, actions, and RTG values from K previous time steps, creating a context window that includes information about the past decisions and environment states. Then, I_t is passed through separate embedding layers for each type of token (`State`, `Action`, `RTG`), resulting in a set of embeddings T_t that feed into the convolution block.

For each convolutional block, it is stacked N times and consists of two sub-blocks:

(1) *Token Mixing* applies the *Layer Normalization* operation followed by a 1D depthwise convolution. A residual connection is added. This captures local temporal transitions between adjacent tokens.

$$Z_1 = \mathrm{Conv}(\mathrm{LN}(T_t)) + T_t \tag{8}$$

where Z_1 is the output of the first sub-block after the 1D convolution operation, $T_t \in \mathbb{R}^{K\times D}$ represents the input sequence of token embeddings at time step t with sequence length K (the context window) and embedding dimension D, $\mathrm{LN}(\cdot)$ denotes Layer Normalization applied along the feature dimension.

The depthwise causal 1D convolution $\mathrm{Conv}(\cdot)$ operates independently on each of the D feature channels. Formally, for each channel $c = 1, \ldots, D$ and output position i,

$$[\mathrm{Conv}(X)]_{i,c} = \sum_{m=0}^{M-1} \mathbf{W}_{c,m} \cdot X_{i-m,c}, \tag{9}$$

where $X = \mathrm{LN}(T_t)$, $\mathbf{W} \in \mathbb{R}^{D\times M}$ are the learnable convolution weights with kernel size M (set to $M = 3$ in our implementation), and the operation is causal: $X_{i-m,c} = 0$ for $i - m < 0$. Z_1 is the output of the first sub-block after the 1D convolution operation.

(2) *Feed-Forward Network (FFN)* processes the normalized output of the first sub-block through an MLP (e.g., with GELU activation), followed by another residual connection. FFN transforms features for decision-making.

$$Z_2 = \mathrm{FFN}(\mathrm{LN}(Z_1)) + Z_1 \tag{10}$$

where Z_2 is the output of the second sub-block, and $FFN(\cdot)$ represents the *feed forward* network, typically a two-layer MLP with a non-linear activation function, GELU.

Formally, for an input $Y = \mathrm{LN}(Z_1) \in \mathbb{R}^{K\times D}$, the FFN is defined as:

$$\mathrm{FFN}(Y) = \left(\mathrm{GELU}\left(Y\mathbf{W}_1 + \mathbf{b}_1\right)\right)\mathbf{W}_2 + \mathbf{b}_2, \tag{11}$$

where $\mathbf{W}_1 \in \mathbb{R}^{D\times D_{\mathrm{ffn}}}$, $\mathbf{b}_1 \in \mathbb{R}^{D_{\mathrm{ffn}}}$, $\mathbf{W}_2 \in \mathbb{R}^{D_{\mathrm{ffn}}\times D}$, and $\mathbf{b}_2 \in \mathbb{R}^{D}$ are learnable parameters, and $D_{\mathrm{ffn}} = 4D$ is the hidden dimension of the FFN.

These operations are repeated over N stacked convolutional blocks (we use $N = 4$). Three separate sets of convolution filters (each with their own $\mathbf{W} \in \mathbb{R}^{D\times M}$) are maintained for the State, Action, and RTG modalities, enabling the model to capture modality-specific temporal patterns while sharing the same FFN and normalization layers across modalities.

To handle the unique dynamics of each modality, three separate convolution filters are used independently across the hidden dimensions: dedicated filters for `State`, `Action`, `RTG` sequences, respectively. This depthwise design allows CDCT-AO actors to learn fine-grained, modality-specific temporal patterns efficiently.

Interaction Mechanism of Modality-Specific Filters. While the three depthwise convolutional filters (for `State`, `Action`, and `RTG` sequences) operate independently on their respective input channels, their

outputs are designed to interact within the subsequent *MetaFormer* block to form a cohesive representation. Specifically, after the modality-specific temporal features are extracted by the separate 1D convolutions, the resulting features are concatenated along the feature dimension. This concatenated representation, denoted as $\mathbf{H}_t = [\mathbf{Z}_2^{\text{state}}, \mathbf{Z}_2^{\text{action}}, \mathbf{Z}_2^{\text{rtg}}]$, serves as the input to the shared channel-mixing MLP within the MetaFormer's feed-forward network (FFN) sub-block. This is the critical point of interaction: the shared MLP learns to mix and re-weight information across all modalities, enabling the model to capture cross-modal dependencies, such as how a past action influenced the subsequent state and the resulting RTG. Thus, the architecture implements a *separate-then-fuse* strategy: specialized filters first capture intra-modal temporal dynamics, followed by a shared non-linear projection that learns inter-modal correlations essential for multi-step credit assignment and return-conditioned policy learning.

Finally, the outputs Z_2 are then processed through the standard Transformer *residual connections* and *layer normalization*. This output becomes the input to the next identical block, enabling iterative feature fusion and transformation across the network's depth.

Overall, the Causal-Convolutional Filter offers three key advantages: *Localized Modeling*, as it excels at capturing *Markovian* dynamics and transitions between consecutive states and actions; *Computational Efficiency*, by replacing the quadratic complexity of attention with a simpler, linear convolution operation; and *Improved Generalization*, where the inductive bias of convolution helps the model perform better on OOD state sequences. Based on this, MA-CCT provides a computationally efficient and effective method for modeling temporal dependencies, making the MA-CCT model particularly suitable for sequential decision-making in multi-agent systems.

4.2. Curiosity-driven exploration mechanism for MARL (CurExp)

To address the critical challenge of limited state coverage and the resulting poor OOD generalization during the online fine-tuning phase of CDCT-AO, we propose a novel intrinsic motivation mechanism named **Curiosity-driven Exploration (CurExp)**. CurExp is designed to incentivize agents to explore environmental states that are underrepresented or entirely missing from the offline dataset, thereby effectively bridging the distributional gap between static pre-collected data and the dynamic online environment.

4.2.1. Curiosity-driven intrinsic rewards calculation of CurExp

The core idea of CurExp is rooted in the concept of *visitation curiosity*: multi-agents are intrinsically rewarded for transitioning to state-action pairs where their predictive models, i.e., actors, of the environment dynamics perform poorly. These prediction errors serve as quantifiable foundations for the agents' uncertainties or novelties about particular state transitions. By pursuing these uncertain transitions, agents are guided toward regions of the state space that lie beyond the support of offline training data, which is crucial for robust adaptation and improved performance on OOD states.

Specifically, CurExp is instantiated as an *Intrinsic Curiosity Module*, which consists of two inverse models and one forward model that operate in a learned feature space, as illustrated in Fig. 4. The feature space is learned by encoder networks, which project raw state observations into a lower-dimensional latent representation where dynamics are potentially simpler to model.

Forward Model ($\mathcal{F}$). This model is an MLP designed to learn the state transition dynamics within the feature space. Specifically, it takes the encoded feature representation of the current state $\phi(s_t)$ and the agents' joint action a_t (in one-hot encoding) as input, and predicts the feature representation of the next global state $\hat{\phi}(s_{t+1})$. The network architecture is formally defined as follows:

$$\hat{\phi}(s_{t+1}) = \mathbf{W}_2^{(f)} \cdot \text{GELU}\left(\mathbf{W}_1^{(f)} \cdot [\phi(s_t); a_t] + \mathbf{b}_1^{(f)}\right) + \mathbf{b}_2^{(f)} \tag{12}$$

where $[\cdot;\cdot]$ denotes the vector concatenation operation. Here, $\mathbf{W}_1^{(f)} \in \mathbb{R}^{(d_{feat}+d_{act})\times h_{fwd}}$ and $\mathbf{W}_2^{(f)} \in \mathbb{R}^{h_{fwd}\times d_{feat}}$ are learnable weight matrices, with d_{feat} being the dimensionality of the feature encoding $\phi(\cdot)$, d_{act} the dimensionality of the joint action space, and h_{fwd} the number of hidden units in the forward model's single hidden layer. $\mathbf{b}_1^{(f)}$ and $\mathbf{b}_2^{(f)}$ are bias vectors. GELU is the activation function. This forward model is optimized via the Mean Squared Error (MSE) loss between its prediction $\hat{\phi}(s_{t+1})$ and the target $\phi(s_{t+1})$ encoded by the feature network ϕ'.

Inverse Model ($\mathcal{I}$). The inverse model is implemented as another MLP. Its role is to ensure that the learned feature encoding $\phi(\cdot)$ retains information necessary for action prediction, thereby promoting the learning of control-relevant features. It takes the concatenated features of the current and next states as input and predicts the joint action $\hat{a}_t$ that caused the transition. Its architecture is defined as:

$$\hat{a}_t = \text{Softmax}(\mathbf{W}_2^{(i)} \cdot \text{ReLU}(\mathbf{W}_1^{(i)} \cdot [\phi(s_t); \phi(s_{t+1})] + \mathbf{b}_1^{(i)}) + \mathbf{b}_2^{(i)}) \tag{13}$$

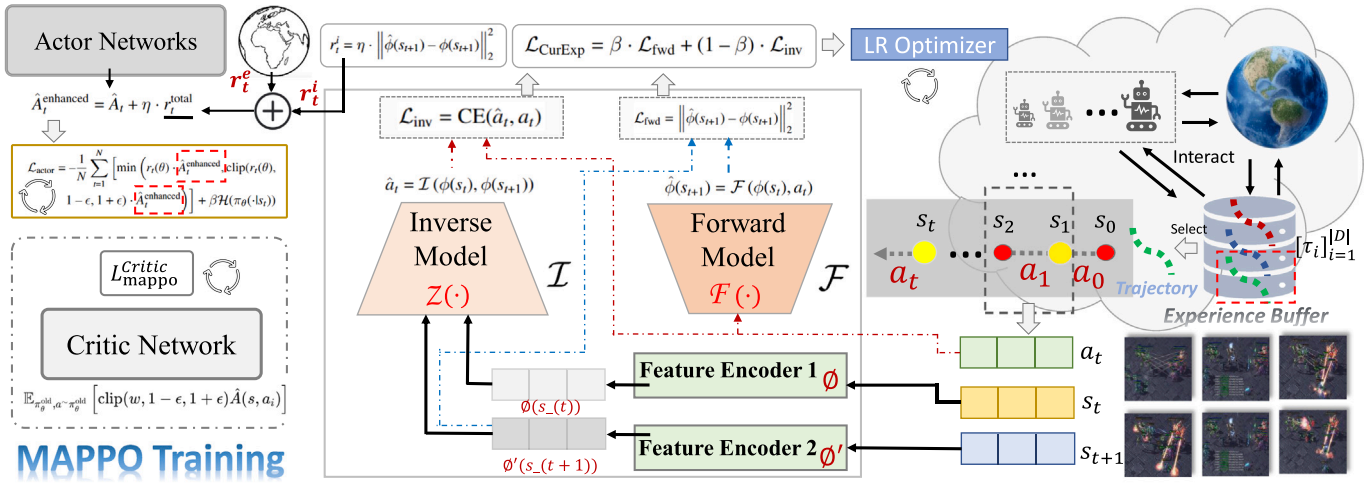


Fig. 4. Architecture and data flow of the Curiosity-driven exploration (CurExp) module. This module takes the current state s_t, executed action a_t, and resulting next state s_{t+1} as inputs. It computes an intrinsic reward r_t^i based on the prediction error of both a forward model and a inverse model. Through this, the two separate encoders (ϕ and ϕ') project states into a latent feature space. **(a) forward Model** $\mathcal{F}$ predicts the next state's features $\hat{\phi}(\mathbf{s}_{t+1})$ from the current features $\phi(\mathbf{s}_t)$ and action $\mathbf{a}_t$. **(b) inverse Model** $\mathcal{I}$ predicts the action $\hat{\mathbf{a}}_t$ from the concatenated current and next state features $[\phi(\mathbf{s}_t), \phi(\mathbf{s}_{t+1})]$. This intrinsic reward is combined with the extrinsic environment reward r_t^e to form the total reward signal r_t^{total} for policy optimization within the MAPPO framework.

Here, $\mathbf{W}_1^{(i)} \in \mathbb{R}^{(2\cdot d_{feat})\times h_{inv}}$ and $\mathbf{W}_2^{(i)} \in \mathbb{R}^{h_{inv}\times d_{act}}$ are learnable weights, with h_{inv} denoting the size of the inverse model's hidden layer. $\mathbf{b}_1^{(i)}$ and $\mathbf{b}_2^{(i)}$ are bias vectors. GELU is the activation function. This model is trained using a cross-entropy loss between the predicted action distribution $\hat{a}_t$ and the true action a_t.

Feature Encoders (ϕ, ϕ'). Two separate neural networks are used to encode the current state s_t and the next state s_{t+1} into the feature space. Notably, the encoder for the next state (ϕ') is dynamically adapted to handle potential dimensional discrepancies between s_t and s_{t+1} (e.g., when global states and local observations are used interchangeably), enhancing the module's flexibility.

Note that both encoders are implemented as separate, shallow CNNs when processing raw pixel observations, or as MLPs for vector-based states. In our experiments with vector states, ϕ and ϕ' are two-layer MLPs with ReLU activations and hidden sizes of 256. While they have identical architectures, their parameters are not shared to provide the flexibility needed to handle potential distributional shifts between s_t and s_{t+1}.

The *intrinsic reward* r_t^i at timestep t is calculated as the *Mean Squared Error* (MSE) between the predicted next-state feature $\hat{\phi}(s_{t+1})$ and the actual encoded next-state feature $\phi(s_{t+1})$, where this error represents the surprise or novelty of the transition.

$$r_t^i = \eta \cdot \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2 \tag{14}$$

where η is a scaling coefficient (*intrinsic_reward_coef*) that controls the influence of the intrinsic reward relative to the extrinsic reward from the environment. The η for the intrinsic reward is set to 0.01. This value ensures that the magnitude of the intrinsic reward r_t^i is appropriately calibrated relative to the extrinsic reward r_t^e provided by the SMAC environment (where episodic returns typically range from 0 to ~20).

Simultaneously, the inverse model is trained via a cross-entropy loss $\mathcal{L}_{\text{inv}}$ to accurately predict the joint action that agents take, ensuring the features learned are relevant for control:

$$\mathcal{L}_{\text{inv}} = \text{CE}(\hat{a}_t, a_t) \tag{15}$$

where $CE(\cdot)$ denotes the '*cross-entropy*' loss function.

The forward model is trained via an MSE loss $\mathcal{L}_{\text{fwd}}$ to improve its predictive capability, defined as follows:

$$\mathcal{L}_{\text{fwd}} = \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2 \tag{16}$$

The total loss for optimizing CurExp is a weighted sum of these two losses, as follows:

$$\mathcal{L}_{\text{CurExp}} = \beta \cdot \mathcal{L}_{\text{fwd}} + (1-\beta) \cdot \mathcal{L}_{\text{inv}} \tag{17}$$

where β is a weighting coefficient (*forward_loss_coef*). The β, controlling the balance between forward ($\mathcal{L}_{\text{fwd}}$) and inverse ($\mathcal{L}_{\text{inv}}$) model losses, is set to 0.2. This allocation prioritizes the inverse model (weight = 0.8), aligning with the core design principle that the feature encoder $\phi(\cdot)$ should primarily learn *control-relevant* representations, which are useful for action prediction.

The loss $\mathcal{L}_{\text{CurExp}}$ is optimized jointly with the MAPPO objective, ensuring that the *Intrinsic Curiosity Network* co-adapts to the agents' evolving state visitation distribution.

4.2.2. Integration with MAPPO-based fine-tuning of CurExp

During the online fine-tuning stage, the intrinsic reward r_t^i generated by CurExp is seamlessly integrated into MAPPO [32]. In specific, the intrinsic reward r_t^i enters the optimization pipeline through Eq. (18), propagates to TD errors (Eq. 20), accumulates into advantages via GAE (Eq. 21), and finally drives policy updates through the PPO objective (Eq. 22).

First, the total reward r_t^{total} used for policy optimization becomes a combination of the extrinsic reward r_t^e provided by the environment and the intrinsic reward r_t^i, which is as follows:

$$r_t^{\text{total}} = r_t^e + \eta \cdot r_t^i \tag{18}$$

where η (*intrinsic_reward_coef*) is a scaling coefficient set to 0.01 in our implementation, and r_t^{total} is then used to compute the enhanced advantage estimates $\hat{A}_t^{\text{enhanced}}$ (e.g., via Generalized Advantage Estimation (GAE) [32,53]) for the policy gradient update.

Consequently, the policy π_θ and the value function V_ϕ are optimized to maximize the expected sum of both extrinsic and intrinsic returns. The integrated reward flows through the following sequential steps in the PPO optimization:

First, the value function V_ϕ is trained to predict the expected total return:

$$V^{\text{target}}(s_t) = r_t^{\text{total}} + \gamma V_\phi(s_{t+1}) \tag{19}$$

The temporal difference (TD) error is calculated as follows:

$$\delta_t = r_t^{\text{total}} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{20}$$

where the δ_t inherently incorporates both extrinsic and intrinsic reward signals.

Then, the enhanced advantage estimates $\hat{A}_t^{\text{enhanced}}$ are computed by exponentially weighting TD errors across timesteps:

$$\hat{A}_t^{\text{enhanced}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{21}$$

where λ is the GAE parameter (typically set to 0.95 in our experiments). This formulation ensures that intrinsic rewards from CurExp properly influence long-term advantage estimates through the recursive accumulation of δ_t.

Then, for the MAPPO-based actor parameter optimization, the policy parameters θ of the actor are optimized by minimizing the standard PPO-Clip objective function $\mathcal{L}_{\text{actor}}$, augmented with an entropy bonus term $\mathcal{H}$ to encourage exploration:

$$\mathcal{L}_{\text{actor}} = -\frac{1}{N}\sum_{t=1}^{N}[\min(r_t(\theta)\cdot \hat{A}_t^{\text{enhanced}}, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\cdot \hat{A}_t^{\text{enhanced}})] + \beta\mathcal{H}(\pi_\theta(\cdot|s_t)) \tag{22}$$

where N is the number of timesteps in the MAPPO mini-batch, $r_t(\theta)$ is the probability ratio between the current policy π_θ and the old policy $\pi_{\theta_{\text{old}}}$ for action a_t at timestep t ($r_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t|s_t)$), the $\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)$ constrains the probability ratio to remain within the interval $[1-\epsilon, 1+\epsilon]$, the ϵ is the clipping hyperparameter set to 0.2 in our implementation, the min operator takes the minimum of the clipped and unclipped objectives, $\mathcal{H}(\pi_\theta(\cdot|s_t))$ is the entropy of the policy distribution, and β is a weighting coefficient of the entropy regularization $\mathcal{H}(\cdot)$.

Finally, the value function is simultaneously optimized via the mean squared error:

$$\mathcal{L}_{\text{critic}} = \frac{1}{N}\sum_{t=1}^{N}\left(V_\phi(s_t) - V^{\text{target}}(s_t)\right)^2 \tag{23}$$

The above formulations provide an explicit mathematical linkage between CurExp's intrinsic rewards and PPO's policy optimization.

Overall, CurExp encourages the agents to explore novel states that yield high prediction errors, leading to more comprehensive state coverage and significantly improved generalization capabilities during fine-tuning, which enhances the final decision-making performance for OOD states of our CDCT-AO framework.

Table 1
The Details of MSAL's phase transition and learning rate Adjustment Mechanism, which are Divided into five distinct stages based on Scheduled win-rate performance evaluation.

Phase	Epoch range	Learning rate adjustment	Win rate feedback	MARL purpose / Effects
WarmUp	[0, `warmup_end`)	Linear increase: base_lr × $\frac{\text{epoch}+1}{10}$ ↑	/	Stabilizes initial learning for both actor and critic networks
Exploration	[`warmup_end, explor_end`) (γ)	Constant boost: base_lr × 1.2 ↑	/	Encourages extensive policy exploration for multiple agents
Normal	[`explor_end`, ∞)	Dynamic adjustment: base_lr × [1.0 ±0.15 × tanh(|0.6-win_rate| × 2)] ↑	**Active monitoring**	Adapts learning pace based on collective performance from both offline and online stages
Fine-tuning	win_rate > `wr_thres`	Reduction: base_lr × 0.5 ↓	**Primary trigger**	Enables precise policy refinement for coordinated multi-agent behaviors
Restart	Every `UP_T` epochs	Jump boost: base_lr × 2.0 ↑	/	Helps escape local optima in complex multi-agent scenarios

4.3. Multi-stage adaptive learning-rate scheduler (MSAL) for MARL optimization

To better control the learning rate of the agent at different stages using real-time feedback from test evaluations, we further designed a novel flexible learning rate scheduler called Multi-Stage Adaptive Learning-rate Scheduler (**MSAL**). MSAL is a sophisticated learning rate optimization module specifically designed for both *offline* and *online* phases of MARL frameworks. In the context of actor-critic architectures, MSAL dynamically adjusts the learning rates for both policy (actor) and value (critic) networks based on real-time performance feedback from both training stages.

As illustrated in Table 1 and Algorithm 1, MSAL operates through multiple distinct phases, each serving a specific purpose in the optimization process. This table demonstrates a five-stage adaptive learning rate strategy where the learning rate undergoes a linear increase during the *WarmUp* phase, maintains elevated levels during the *Exploration* phase, dynamically oscillates based on real-time performance in the *Normal* phase, strategically decreases during the *Fine-tuning* phase, and periodically resets with amplified values in the *Restart* phase, creating a cyclic optimization pattern that balances exploration and exploitation throughout the training process.

Specifically, MSAL continuously monitors the win rate through a sliding window of recent outcomes (default window size: 100 episodes) from both offline and online stages. This metric serves as a comprehensive indicator of the overall effectiveness of the current policies. For both offline and online actor networks, MSAL adaptively balances exploration-exploitation tradeoffs. Higher learning rates during exploration phases promote diverse policy sampling, while fine-tuning phases enable precise behavioral refinement. For critic networks, adaptive adjustments ensure proper value function updates that reflect multi-agent policy changes during behaviour imitation and environment interaction.[4]

To be specific, as shown in Algorithm 1, which provides the formal pseudocode. The core symbols used in the MSAL scheduler are defined as follows:

- `base_lr`: the base learning rate (e.g., 1×10^{-4});
- `warmup_end` and `explor_end`: epoch indices marking the transitions from WarmUp to Exploration and from Exploration to Normal phases, respectively;
- `wr_thres`: the win-rate threshold (0.6) for triggering the Fine-tuning phase;
- `UP_T`: the fixed epoch interval (e.g., 200) for triggering the Restart phase;
- `win_rate`: the average success rate computed over a sliding window of the last 100 evaluation episodes.

[4] In our experiments, the learning rate adjustments are constrained within predefined bounds $[10^{-5}, 10^{-3}]$ to ensure training stability throughout both training phases.

Input: base_lr, wr_thres, UP_T, epoch, warm- up_end, explor_end, win_history;
Output: dynamic lr (current learning rate);

```
# 1. Compute current win-rate from sliding window
if len(win_history) s= 100:
    win_rate = mean(win_history[−100:])
else:  win_rate = 0.0
# 2. Determine base learning rate multiplier
if epoch % UP_T = = 0: # Restart Phase
    multiplier = 2.0
elif epoch < warmup_end: # WarmUp Phase
    multiplier = (epoch + 1) / 10.0
elif epoch < explor_end: # Exploration Phase
    multiplier = 1.2
elif win_rate > wr_thres: # Fine-tuning Phase
    multiplier = 0.5
else: # Normal Phase
    error = 0.6 - win_rate
    adjustment = tanh(abs(error) * 2) * 0.15
    multiplier = 1.0 + sign(error) * adjustment
# 3. Calculate, clamp, and return the final learning rate
lr = base_lr * multiplier
lr = clamp(lr, min=1e-5, max=1e-3)
return lr
```

Next, we conduct a detailed analysis of these dynamic stages.

- **WarmUp** ([0, `warmup_end`)): The learning rate increases linearly from a small value to the `base_lr`, stabilizing early training.
- **Exploration** ([`warmup_end`, `explor_end`)): The learning rate is boosted to a constant high value (`base_lr` × 1.2) to encourage diverse policy exploration.
- **Normal** ([`explor_end`, ∞)): The learning rate is dynamically adjusted according to the current win-rate (`win_rate`) to adapt the learning pace. It increases if performance is below the threshold (`wr_thres` = 0.6) and decreases if above, following a smoothed, bounded function.
- **Fine-tuning** (Triggered when `win_rate` ≥ `wr_thres`): This phase overrides the Normal phase, reducing the learning rate by 50% (`lr` × 0.5) to enable precise policy refinement near a performance plateau.
- **Restart** (Triggered every `UP_T` epochs): This phase periodically overrides others, jumping the learning rate to `base_lr` × 2.0 to help the optimizer escape potential local minima.

The `win_rate` is computed as the average success rate over a sliding window of the most recent evaluation episodes (default size: 100). All learning rates are clipped to the range $[10^{-5}, 10^{-3}]$ for stability.

Table 2
Statistical Overview of MAPPO [32]-generated expert demonstrations via *Meng et al.*'s MADT work [27]. `#Samples` denotes the number of POMDP transitions (per timestep) in a specific map. `Total return (Mean (±Std))` represents the average cumulative reward, i.e., return, (with standard deviation) obtained per episode in the offline trajectories.

Difficulty	Maps	#Samples	Total return (Mean (±Std))	Difficulty	Maps	#Samples	Total return (Mean (±Std))
Easy	*3m*	1 775 159	19.99 (±2.17)	**Easy**	*3s vs.5z*	2 604 082	20.89 (±1.38)
	8m	2 781 145	20.00 (±0.16)	**Hard**	*3s5z*	1 148 424	18.45 (±2.03)
	2s vs.1sc	1 972 972	20.23 (±0.02)		*10m vs.11m*	895,609	20± (0.01)
	2m vs.1z	120 127	20.00 (±0.01)		*5m vs.6m*	503 746	20 (±0.01)
	2s vs.3z	4 177 846	19.93 (±0.67)		*MMM*	2 423 605	20.08 (±1.67)
	3s vs.4z	3 080 634	20.19 (±0.40)				
	bane vs.bane	458 795	19.97 (±0.36)	**Super-Hard**	*3s5z vs.3s6z*	1 542 571	18.35 (±2.04)
	So_many_baneling	1 376 861	19.46 (±1.29)		*corridor*	3 163 243	19.88 (±0.99)

Table 3
The details of experimental hyper-parameters of our proposed CDCT-AO framework.

Hyper-parameters in CDCT-AO					
Name	Value	Name	Value	Name	Value
actor_input_type	state_only	max_timestep	400	MA-CCT_layer	2
critic_offline_train	True	critic_online_train	True	feature_dim	128
offline_eval_interval	1 epoch	online_eval_interval	32 episodes	curiosity_lr	1e-4
offline_episode_num	[200,1000]	offline_lr	5e-4	forward_loss_coef	0.2
online_epochs	5000	online_lr	1e-4	inverse_loss_coef	0.1
ppo_clip_param	0.2	online_ppo_epochs	[5,10,15]	intrinsic_reward_coef	0.01
offline_buffer_size	64	online_buffer_size	64	curiosity_decay	0.99
win_rate_threshold	0.6	actor_lr_bounds	(1e-5, 1e-3)	critic_lr_bounds	(1e-4, 5e-4)
offline_mini_batch_size		128	exploration_epochs	20	
UP_T	200				
warmup_end	10	exploration_end	50	wr_threshould	0.8

The default values for all hyperparameters in our MSAL module listed in Table 1 can be found in Table 3.

The integration of MSAL within CDCT-AO demonstrates significant improvements in offline-to-online adaptation, particularly in handling OOD states and bridging the gap between pre-trained policies and online environment interactions. By providing adaptive, performance-driven learning rate adjustments that leverage feedback from both training stages, MSAL enables more robust optimization of CDCT-AO's complex multi-agent policies.

5. Experiments

In this section, we systematically conduct performance validation for our CDCT-AO framework. Section 5.1 introduces the experimental settings, and Section 5.2 evaluates CDCT-AO against SOTA baselines across multiple map scenarios, presenting comparisons in terms of average return, win-rate, and quantized best-win-rate results for purely offline scenarios and online fine-tuning. In Section 5.3, we conduct a series of quantitative ablation studies demonstrating the effectiveness of each submodule in our CDCT-AO framework.

5.1. Experimental settings

This part outlines the experimental setup, covering the SMAC benchmark environment, offline data statistics, performance evaluation metrics, SOTA baseline algorithms for offline MARL, model hyperparameter settings, and hardware specifications.

5.1.1. StarCraft-II SMAC benchmark

Our experimental evaluation utilizes the StarCraft Multi-Agent Challenge (SMAC) [54], which provides a comprehensive framework for assessing decentralized control strategies in complex combat scenarios.

This SMAC benchmark has become a standard platform for evaluating cooperative multi-agent reinforcement learning algorithms due to its realistic partial observability constraints and demanding coordination requirements. Developed as an extension of the StarCraft II game engine,[5] SMAC[6] [54,55] simulates tactical micromanagement scenarios where heterogeneous agent teams must collaborate under strict observation limitations. The platform's emphasis on decentralized control with partial observability makes it particularly relevant for real-world applications such as disaster rescue [9,10], autonomous driving [11,13], multi-robot production logistics systems [7,8], and other distributed robotic systems requiring robust multi-agent decision-making.

The SMAC environment presents a challenging partially observable Markov decision process (POMDP) framework, where agents must execute decentralized actions, such as movement, attacks, and special abilities, based solely on localized observations. With heterogeneous agent capabilities requiring specialized roles, teams must develop sophisticated coordination strategies despite limited observation and attack ranges that prevent global awareness. The dynamic adversarial conditions demand real-time tactical decision-making under strict resource constraints, with the primary objective being to eliminate all opposing units within a fixed time window for substantial terminal rewards. These features create a complex strategic environment that tests multi-agent cooperation under uncertainty, mirroring real-world challenges in distributed autonomous systems.

To illustrate the environmental diversity, Fig. 5 showcases several representative battle scenarios ranging from basic symmetric confrontations to complex asymmetric engagements. These scenarios test different aspects of multi-agent coordination, including unit composition, terrain utilization, and tactical positioning.

5.1.2. Baselines

To ensure fair and comprehensive comparisons, we implement several SOTA offline MARL algorithms as competitive baselines. These methods represent different approaches to addressing the key challenges

[5] PySC2 interface: https://github.com/google-deepmind/pysc2
[6] SMAC repositories: https://github.com/oxwhirl/smac and https://github.com/oxwhirl/smacv2

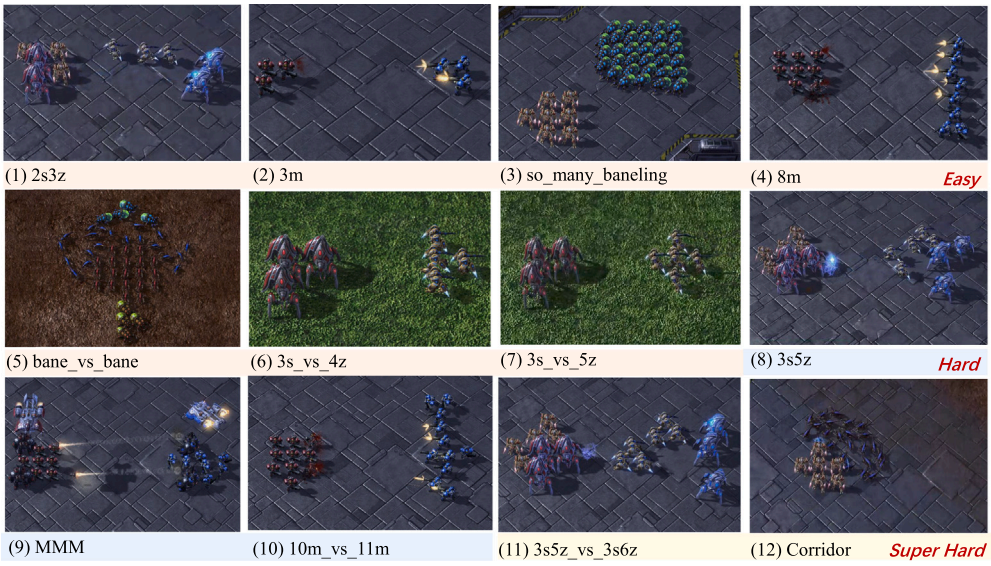


Fig. 5. Twelve representative SMAC battle scenarios adopted in our experiments categorized by difficulty level (*Easy*, *Hard*, and *Super-Hard*), demonstrating the progressive complexity in unit composition and tactical requirements for MARL.

in offline MARL, such as out-of-distribution actions, multi-agent credit assignment, and offline policy optimization. The selected baselines include:

- **BC**: A multi-agent adapted version of the *Behavioral Cloning* method [31], which uses bi-level optimization to achieve value-aligned behavior cloning. This approach has demonstrated reliable performance in single-agent offline RL by directly mimicking the behavior policy without explicit reward optimization.
- **BCQ-MA**: The multi-agent extension of Batch-Constrained Q-learning (BCQ) [20,51], which constrains the action space to prevent the selection of out-of-distribution actions. BCQ-MA extends this approach to multi-agent settings through decentralized value networks while maintaining batch-constrained policy optimization.
- **ICQ-MA**: The multi-agent version of Implicit Constraint Q-learning [18], which adopts a "believe what you see" philosophy by implicitly constraining policy updates to stay close to the behavior policy. This method shows particular effectiveness in handling the distributional shift problem in offline RL.
- **CQL-MA**: A multi-agent adaptation of Conservative Q-learning [16], which learns a conservative Q-function that lower-bounds the true value function to prevent overestimation of out-of-distribution actions. The multi-agent extension maintains this conservative property while addressing credit assignment among agents.
- **MADT**: The Offline Pre-trained Multi-agent Decision Transformer [27], which extends the Decision Transformer architecture to multi-agent settings. It models the sequential decision-making process using transformer architectures and has shown promising results in offline multi-agent scenarios.

These baselines represent three major categories of approaches in offline multi-agent RL: (1) policy constraint methods (BCQ-MA, ICQ-MA), (2) value regularization approaches (CQL-MA), and (3) sequence modeling techniques (MADT). BC serves as a simple but strong baseline that directly learns from the offline dataset without considering the reward signal during training.

The multi-agent extensions of single-agent algorithms (BCQ-MA, ICQ-MA, CQL-MA) are implemented through straightforward integration of individual agents' value networks while preserving their original theoretical guarantees. Among them, MADT represents a fundamentally different approach based on recent advances in transformer architectures for decision making.

5.1.3. Offline data: expert-level trajectory demonstration

To ensure comparison rigor, our training leverages trajectories generated by MAPPO [32] and then processed by MADT [27].[7]

Table 2 summarizes key statistics of this dataset, which comprehensively covers most SMAC battle scenarios. The collection contains tens of thousands of episode trajectories $\{s_t, o_t, \mathbf{a}_t, r_t, \text{done}_t, s_{t+1}\}_{t=0}^{T-1}$ (approximately millions of MDP transitions $\{s_t, o_t, \mathbf{a}_t, r_t, \text{done}_t, s_{t+1}\}_t$ at timestep t). Consistent with Dec-POMDP principles, agents in this dataset make decisions exclusively through partial observations o_t rather than full state information s_t.

5.1.4. Evaluation metric

Our experimental evaluation employs three widely-used performance metrics that collectively capture different dimensions of agent capabilities in SMAC scenarios.

- **Mean Episode Return**: The mean episode return serves as the fundamental measure of overall policy effectiveness, calculating the undiscounted sum of rewards obtained throughout complete battle episodes. In the context of StarCraft micromanagement tasks, this metric reflects the agents' ability to maintain strategic advantages while minimizing unit losses during combat engagements.
- **Mean Win Rate**: The win rate metric provides a more direct assessment of tactical success, measuring the percentage of episodes where the allied forces achieve complete elimination of enemy units. This binary outcome metric is particularly meaningful in SMAC as it aligns with the ultimate military objective of the simulated battles.
- **Mean of Top-20 Highest Returns**: To complement these conventional measures, we further introduce *Mean of Top-20 Highest Returns* as a specialized metric for evaluating policy robustness during the fine-tuning phase. By averaging the top 20 highest returns observed during evaluation periods, i.e., $R_{\text{highest}} = \frac{1}{20}\sum_{i=1}^{20} r_{(i)}$ where $r_{(i)}$ denotes the i-th highest return, this metric emphasizes

[7] Publicly accessible at: https://github.com/ReinholdM/Offline-Pre-trained-Multi-Agent-Decision-Transformer

consistent high-performance capability rather than average behavior. The statistics of Top-20 returns avoid the interference of potentially lower returns in the early exploration stage or the final convergence stage, and also reduce the random fluctuations brought about by individual exceptionally high-return rounds. The selection of Top-20 is based on our training scale (millions of steps) and evaluation frequency (evaluated every few thousand steps), ensuring a balance between high-performance stages and statistical stability.

All experimental results reported in this paper are obtained by averaging over five fixed independent runs with different random seeds to ensure robustness and reproducibility. The learning curves presented in the successive figures report the mean performance across runs, while the shaded regions indicate the corresponding standard deviation, reflecting the variability and stability of each method during both offline pre-training and online fine-tuning. All metrics are computed over five independent experimental runs per method-map combination and accompanied by 85% confidence intervals to ensure statistical reliability.

5.1.5. Experimental settings

Model Implementation Details: The comprehensive evaluation of our CDCT-AO framework spans all critical components: (1) the architectural design of MA-CCT that incorporates both causal-convolution filters and Metaformer, (2) map scenario configurations, (3) offline pre-training, and (4) online fine-tuning (including MAPPO optimization details) phases. The parameter choices reflect careful balancing between model capacity and training stability requirements specific to decentralized multi-agent control tasks.

Offline Dataset Size and Random Seeds. For each map, the offline dataset consists of 1000 expert-level episodes collected using MAPPO, resulting in a fixed and reproducible offline dataset size per scenario. To reduce stochastic bias and ensure reproducibility, all reported results are averaged over multiple random seeds. Specifically, we use three fixed random seeds: {1, 123, 321}, which are consistently applied across offline pre-training, online fine-tuning, and baseline comparisons.

Hardware Settings: Our implementation of **CDCT-AO** is built on PyTorch 1.12.1[8] and Python 3.8.5,[9] deployed on an Ubuntu 20.04.1 operating system. The experimental platform consists of: (1) Four *Intel Xeon Platinum 8488C* processors (48 cores each); (2) Four *NVIDIA GeForce RTX 4090* GPUs for accelerated computation. For comprehensive implementation specifics, including environment configuration, model architecture, and training protocols, please refer to our provided open-source project.[10]

Pure Online Baseline Configuration: To ensure methodological rigor in the offline-to-online paradigm comparison, we configured a strong and consistent online learning baseline. For the baseline algorithm, we selected MAPPO [32] as our pure online baseline. MAPPO is a widely recognized on-policy algorithm that has demonstrated strong performance on SMAC.

For a fair comparison of network architecture, we ensured architectural parity between the pure online MAPPO and the online fine-tuning phase of our CDCT-AO. Specifically:

[1]. The actor network in both configurations is a 2-layer GPT-based model (i.e., `n_layer=2`) with an embedding dimension of 32 (i.e., `n_embd=32`).

[2]. The centralized critic network is also a 2-layer GPT with identical dimensions.

For a fair comparison of offline-to-online hyperparameters, the core hyperparameter configurations during online-tuning were also set to align with established best practices.

[1]. The hyperparameters for the online MAPPO baseline were set to align with established best practices for training on SMAC: (1) a learning rate (online_lr) of 5e-4, (2) a PPO clip range (ppo_clip_param) of 0.2, (3). a discount factor (gamma) of 0.99, (4) a entropy coefficient of 0.01, and (5). a hidden dimension of 128.

[2]. The pure online baseline was trained from scratch (with randomly initialized network weights) for 5000 epochs (`online_epochs=5000`). Each epoch involves collecting a batch of experience from the environment using the current policy (with a `online_buffer_size=64`), and performing 10 PPO update epochs (`online_ppo_epochs=10`).

The above rigorous settings ensure methodological fairness: we adopt MAPPO as the standard pure online baseline, align all network architectures for an apples-to-apples comparison, and strictly match both hyperparameters and the total online interaction budget between the pure online training and our fine-tuning phase.

5.2. Main results

In this part, we comprehensively assess the performance of CDCT-AO in both offline pre-training and online adaptation phases by comparing it with several representative offline MARL methods. Through systematic comparisons with five representative baselines (BCQ-MA, CQL-MA, IQL-MA, BC, and MADT), we employed three quantitative metrics, i.e., *mean episode return*, *mission success rate*, and *mean of Top-20 highest returns performance*, to provide a multi-dimensional assessment of CDCT-AO's advantages.

The evaluation covers the full spectrum of SMAC challenge levels across 15 distinct battle scenarios, including **9 Easy Maps**: `3m`, `8m`, `2m vs.1z`, `2s vs.1sc`, `2s vs.3z`, `3s vs.4z`, `3s vs.5z`, `So_many_baneling`, and `bane vs. bane` representing standard combat situations; **4 Medium-Hard Maps**: `3s5z`, `10m vs.11m`, `5m vs.6m`, and `MMM` featuring complex unit compositions; and **2 Super-Hard Maps**: `corridor`, and `3s5z vs.3s6z` presenting extremely complex tasks.

5.2.1. Online fine-tuning performance

To demonstrate CDCT-AO's superior performance compared to state-of-the-art baselines in online fine-tuning scenarios, we conducted comprehensive experiments across 12 challenging SMAC maps spanning three difficulty levels. To ensure a rigorous comparison, all baseline methods undergo identical fine-tuning procedures with MAPPO-based policy updates.

Fig. 6 demonstrates the evolution of episode-wise returns during fine-tuning, where all methods benefit from substantial initial returns due to offline pre-training. This validates both the quality of our expert trajectories and the efficiency of the offline-to-online paradigm compared to pure online learning.

First, what we can readily observe from the table is: Conventional methods exhibit characteristic limitations: BCQ-MA and CQL-MA show steady but constrained improvement (e.g., from 14.5 to 18.5 on `bane vs. bane`), while ICQ-MA and BCQ-MA struggle on complex maps (from 8.5 to 13.5 on `MMM`). These patterns reflect their fundamental challenges in handling distributional shift during adaptation. In contrast to these baselines, our CDCT-AO consistently achieves superior performance across all scenarios, demonstrating three notable advantages. On the `bane vs. bane` map, it attains the highest returns exceeding 20, significantly outperforming baseline methods that plateau at 18~19. The framework shows particularly robust generalization capabilities on the `3s5z` scenario, maintaining an averaged return of 22 compared to MADT's 20.5, while also exhibiting exceptional stability with low variance across relatively simpler maps like `3m`, `8m`, and `2s3z`.

To ensure full reproducibility, we systematically list all implementations as follows:

[8] PyTorch: https://pytorch.org/get-started/

[9] Python: https://www.python.org/

[10] The released CDCT-AO Code: https://github.com/albert-jin/CDCT-AO *[Accessed on 2025.08]*

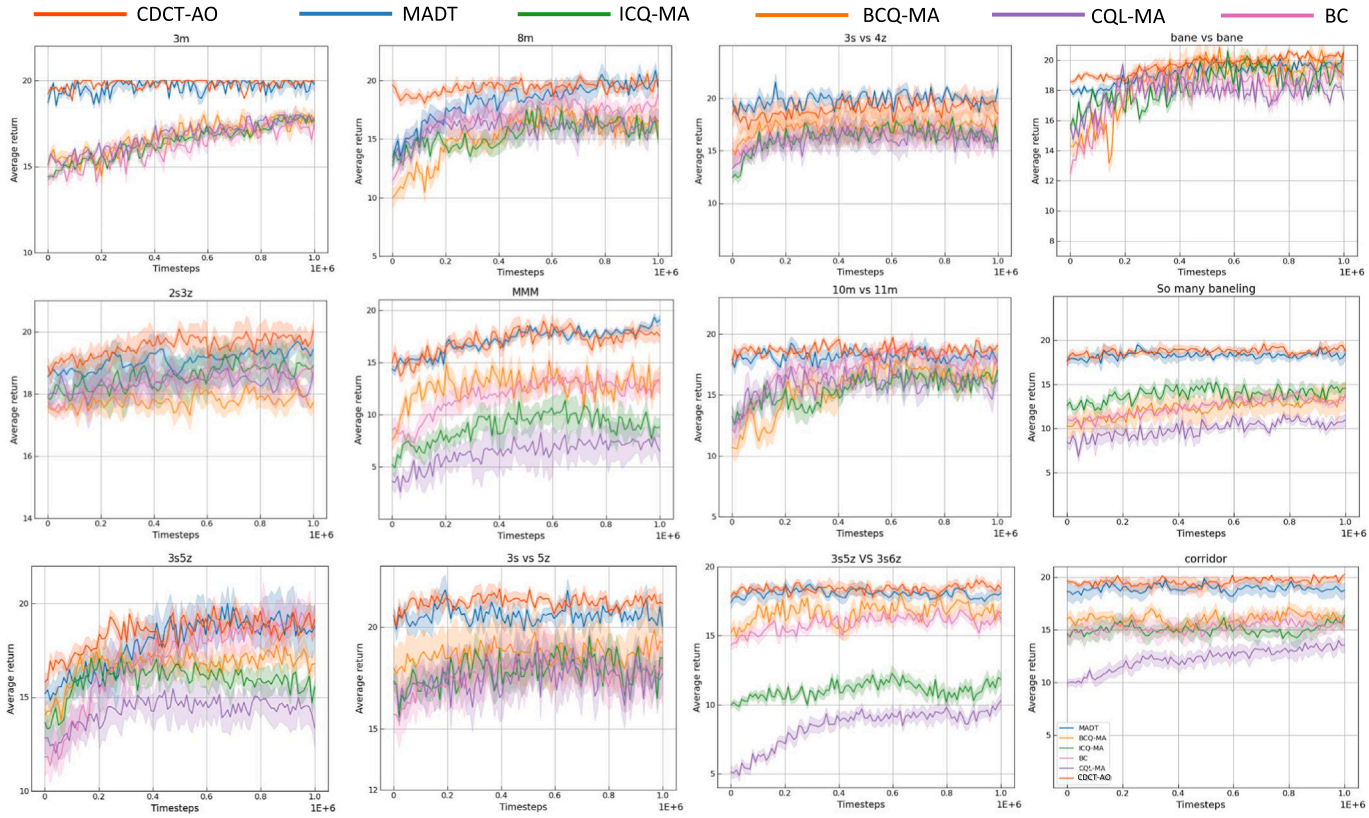


Fig. 6. Online fine-tuning performance comparison: Our CDCT-AO against baselines, evaluated via real-time tracking of mean episode returns across 12 representative SMAC maps. Here, the `x-axis` denotes training timesteps during online fine-tuning, and the `y-axis` represents the average episode return. All baselines undergo online fine-tuning under the MAPPO framework, initializing from their actor networks pre-trained in the offline phase. All reported results are averaged over a fixed 5 independent runs with different random seeds. Shaded areas indicate the standard deviation across runs.

These performance gains directly stem from CDCT-AO's three key architectural innovations working in concert. The MA-CCT architecture's causal-convolution token mixer fundamentally improves upon conventional attention mechanisms by precisely capturing localized temporal patterns that are critical for modeling MDP dynamics. Complementing this, the CurExp mechanism actively bridges the offline-online distribution gap visible in Fig. 6 by strategically directing exploration toward high-prediction-error states. Meanwhile, the MSAL scheduler dynamically maintains optimization stability through its adaptive learning-rate adjustments, proving particularly crucial in challenging map configurations like `corridor` and `3s5z vs.3s6z`.

While both CDCT-AO and MADT jointly dominate other baselines, confirming the general effectiveness of Transformer-based architectures for offline MARL, CDCT-AO's specialized design yields consistent 1.5∼2.5 point advantages on mean episode return by more effectively addressing three fundamental challenges: it better models local temporal dependencies through its causal convolutions, achieves more reliable OOD generalization via curiosity-driven exploration, and maintains superior training stability through its adaptive optimization mechanism.

5.2.2. Statistics of mean of top-20 highest returns

For a comprehensive evaluation of CDCT-AO's performance boundaries, we conduct statistical analysis of mean highest returns during online fine-tuning. The mean highest returns are obtained by calculating the average of the top-20 highest returns as a standardized metric,[11] which reveals CDCT-AO's performance ceiling across different SMAC scenarios.

As evidenced in Table 4, CDCT-AO demonstrates remarkable consistency, outperforming baselines in 11 of 12 maps while maintaining competitive performance in the remaining scenarios. It achieves particularly significant improvements on complex maps, delivering a **0.90**-point gain on the moderately difficult `So_many_baneling` scenario and a **1.30**-point enhancement on `corridor` compared to MADT. Notably, CDCT-AO reaches the theoretical maximum return of 20.00 on fundamental maps like `3m`, `8m` and `bane vs. bane`, while still pushing performance boundaries in near-optimal scenarios. Simultaneously, CDCT-AO evidences its superiority with its **0.39**-point advantage over CQL-MA on `2s3z` where most methods approach the highest returns. The few marginally subpar performances, e.g., a decrease of **0.12**-point on `3s5z`, remain within error thresholds, suggesting stable optimization rather than algorithmic limitations.

Overall, these results firmly validate CDCT-AO's dual capability: achieving theoretical performance ceilings where possible while significantly advancing the state-of-the-art in challenging scenarios through its innovative architecture and optimization mechanisms.

5.2.3. Sample efficiency analysis

To quantify CDCT-AO's practical efficiency gains, we analyze the environmental interactions required to achieve progressive win rate thresholds (20% to 100%) across 12 SMAC scenarios. Specifically, in this section, we compare our *offline pre-training + online fine-tuning* paradigm against pure online learning (denoted by '–' where convergence fails),

[11] Under our specific experimental setup, the calculation of Top-20 average returns corresponds to the optimal performance in the return distribution, which is consistent with common studies evaluating "peaks" or "best performance intervals" (such as "best average win rate"). Essentially, it is a truncated mean that focuses on the right tail of the distribution.

Table 4
Statistical Comparative results of Top-20 highest average Returns, i.e., *Mean of Top-20 highest Returns*, during online Fine-Tuning Process. *(Bold numbers indicate the best performances; Underlined numbers denote the second-best performances; and Parentheses indicate improvement over the top baseline)*.

SMAC map scenarios	CDCT-AO $_{(ours)}$	MADT	BCQ-MA	ICQ-MA	BC	CQL-MA
3m	**20.00** (+0.00)	**20.00**	17.86	17.69	17.51	17.83
8m	**20.01** (+0.12)	19.89	17.21	16.89	18.30	16.77
2s3z	**19.66** (+0.39)	19.07	17.95	18.22	17.30	19.27
MMM	**18.33** (−0.07)	18.40	14.32	10.42	13.39	7.65
3s5z	**18.87** (−0.12)	18.99	15.46	15.06	14.30	10.94
3s vs.5z	**21.60** (+0.44)	21.16	19.31	18.83	18.47	18.47
3s vs.4z	19.86 (−0.83)	**20.69**	17.91	17.50	17.07	17.14
10m vs.11m	**19.27** (+0.51)	18.76	17.56	16.82	18.54	16.98
bane vs.bane	**20.34** (+0.10)	19.91	20.24	19.79	19.38	18.86
So_many_baneling	**19.22** (+0.80)	18.78	13.28	14.84	13.67	11.05
3s5z vs.3s6z	**18.46** (+0.40)	18.06	17.09	11.86	15.82	5.42
corridor	**19.61** (+1.30)	18.31	15.71	15.06	13.03	9.68

as shown in Table 5. The reported '# Samples' refers to the cumulative count of environment steps (where all agents act simultaneously per step). The values represent the point at which the policy's moving average win rate, computed over a window of the 100 most recent evaluation episodes, first meets or exceeds the given threshold. The pure online baseline (MAPPO) was trained from scratch with an identical network architecture and a matched maximum training budget of 2.5 million steps.

In easy scenarios (`2m vs.1z`, `3m`, `8m`), CDCT-AO demonstrates rapid convergence, achieving near 100% win rates with just 1.5×10^5 to 8.4×10^5 samples, while pure online learning fails completely on 7 of 8 easy-map thresholds. The advantage is most pronounced in early learning stages on `3m`, CDCT-AO reaches near 20% win rate with merely 2.5×10^3 samples, indicating immediate policy usability after pre-training.

For hard maps (`3s5z`, `10m vs.11m`), CDCT-AO maintains significant advantages, particularly at higher performance thresholds. On `3s5z`, it achieves a 60% win rate with 4×10^5 samples – a 4.5× reduction compared to pure online learning (1.8×10^6). The `3s vs.5z` scenario showcases CDCT-AO's smooth transition capability: while both methods eventually reach a 100% win rate, CDCT-AO requires 10.7× fewer samples (1.5×10^5 vs. 1.6×10^6).

In super-hard scenarios (i.e., `corridor`, `3s5z vs.3s6z`), CDCT-AO's architectural advantages become decisive. While pure online learning fails to achieve even 20% win rate on `3s5z vs.3s6z`, CDCT-AO reaches 60% with 8.2×10^5 samples and 80% with 1×10^6 samples. The extreme case of `corridor` demonstrates CDCT-AO's persistence: it eventually achieves 100% win rate after 4×10^5 samples where pure online learning plateaus at 6.4×10^6 samples (16× difference).

To provide a comprehensive sample efficiency improvement, we computed the ratio of sample requirements between pure online learning and CDCT-AO for each map and each win-rate threshold (where both methods achieved the target). The median improvement across all thresholds and maps is around 3× to 5×, with the 25th and 75th percentiles being 2.8× and 9.7×, respectively. The maximum observed improvement (near 16×) occurs in the most challenging `corridor` scenario at 100% win rate, while the minimum (1.5×) appears in simpler scenarios like `2m vs.1z`.

In summary, these results validate that CDCT-AO's integration of MA-CCT architecture, CurExp exploration, and MSAL optimization collectively enables superior sample efficiency.

5.2.4. Offline performance evaluation of CDCT-AO

To validate CDCT-AO's capability in addressing core offline MARL challenges, particularly distributional shift and limited state coverage, we conduct a controlled comparative experiment measuring pure offline learning performance without online fine-tuning influences. As illustrated in Fig. 7, we compare CDCT-AO's MA-CCT actor against baseline methods (BC, CQL-MA, ICQ-MA, and MADT) across 10~100 training epochs under the pure offline pre-training phases. The evaluation protocol employs expert demonstration datasets of 200 episodes per scenario (i.e., *3s_vs_4z, 2m_vs_1z, 3m, 2s_vs_1sc, 3s_vs_3z* and so on) for training, with performance evaluated every epoch over 10 test episodes. Training terminates after 10 epochs using a batch size of 128 and a learning rate of 5e-4, ensuring consistent comparison across methods.

The results demonstrate CDCT-AO's superior performance in handling fundamental offline learning limitations. Unlike conventional methods that exhibit progressive divergence from behavior policies, CDCT-AO maintains stable policy alignment throughout training. On challenging maps like `3s vs.5z` and `corridor`, CDCT-AO achieves 15~20% higher average returns compared to MADT, with particularly notable advantages emerging after initial epochs, where distributional shift typically exacerbates value estimation errors.

Notably, baseline methods such as BC and ICQ-MA exhibit rapid performance saturation because they struggle to generalize beyond the

Table 5
Statistics of the sample efficiency comparison, i.e., the number of environment steps required to achieve target win rates during online fine-tuning. We report steps for our CDCT-AO (left of "/") versus a pure online MAPPO baseline trained from scratch (right of '/'). '-' indicates the baseline failed to achieve the target win rate within the maximum training budget of 2.5×10e6 steps. '∞' denotes that neither method achieved the target. Win rate thresholds are considered achieved when the average win rate over the most recent 100 evaluation episodes reaches or exceeds the specified value.

Maps	# Samples to achieve the win rate					Maps	# Samples to achieve the win rate				
	20%	40%	60%	80%	100%		20%	40%	60%	80%	100%
2m vs.1z (Easy)	6.7E+4/–	8.8E+4/–	1E+5/–	1.4E+5/–	1.5E+5/–	3s vs.5z (Hard)	7.5E+5/–	8E+5/–	8.4E+5 /1.5E+4	8.6E+5 /5E+4	1.6E+6 /1.5E+5
3m (Easy)	2.5E+3/–	6.8E+4/–	2.5E+5/–	3.6E+5/–	6.4E+5/–	3s5z (Hard)	7.8E+5 /6.2E+4	1.6E+6 /1.5E+5	1.8E+6 /4E+5	2E+6 /1E+6	2.5E+6 /2E+6
8m (Easy)	4E+5/–	4.5E+6/–	5E+5/–	5.5E+ 5 /1.5E+5	8.4E+5 /2E+5	10m vs.11m (Hard)	4E+4/–	5E+4/–	1E+5 /2.5E+4	1E+6 /1.5E+5	2E+6 /2.5E+5
3s vs.4z (Easy)	3.2E+5/–	4E+5/–	5.5E+5/–	6.3E+5/–	1.5E+6/-	5m vs.6m (Hard)	1.8E+6 /2.5E+5	2.8E+6 /8E+5	5.4E+6 /2E+6	∞/∞	∞/∞
2s vs.1sc (Easy)	1.2E+4/–	2.4E+4/–	2.8E+4/–	6.8E+4 /4E+4	3.1E+5 /1.2E+5	MMM (Super hard)	5.5E+4/–	8.4E+4/–	3.6E+5/–	4.5E+5/–	2E+6/-
So many baneling (Easy)	2.5E+4 /8E+3	1E+5 /4.3E+4	3.4E+5 /6.8E+4	4.5E+5 /7.5E+4	1E+6 /6E+5	3s5z vs.3s6z (Super hard)	1.5E+6/–	2.5E+6/–	3E+6 /8.2E+5	4.5E+6 /1E+6	∞/∞
bane vs.bane (Easy)	3E+3/–	3.4E+3/–	3.2E+5/–	4.2E+5/–	5.2E+5/–	corridor (Super hard)	1.4E+6/–	1.3E+6/–	1.6E+6/–	2.4E+6/–	6.4E+6 /4E+5

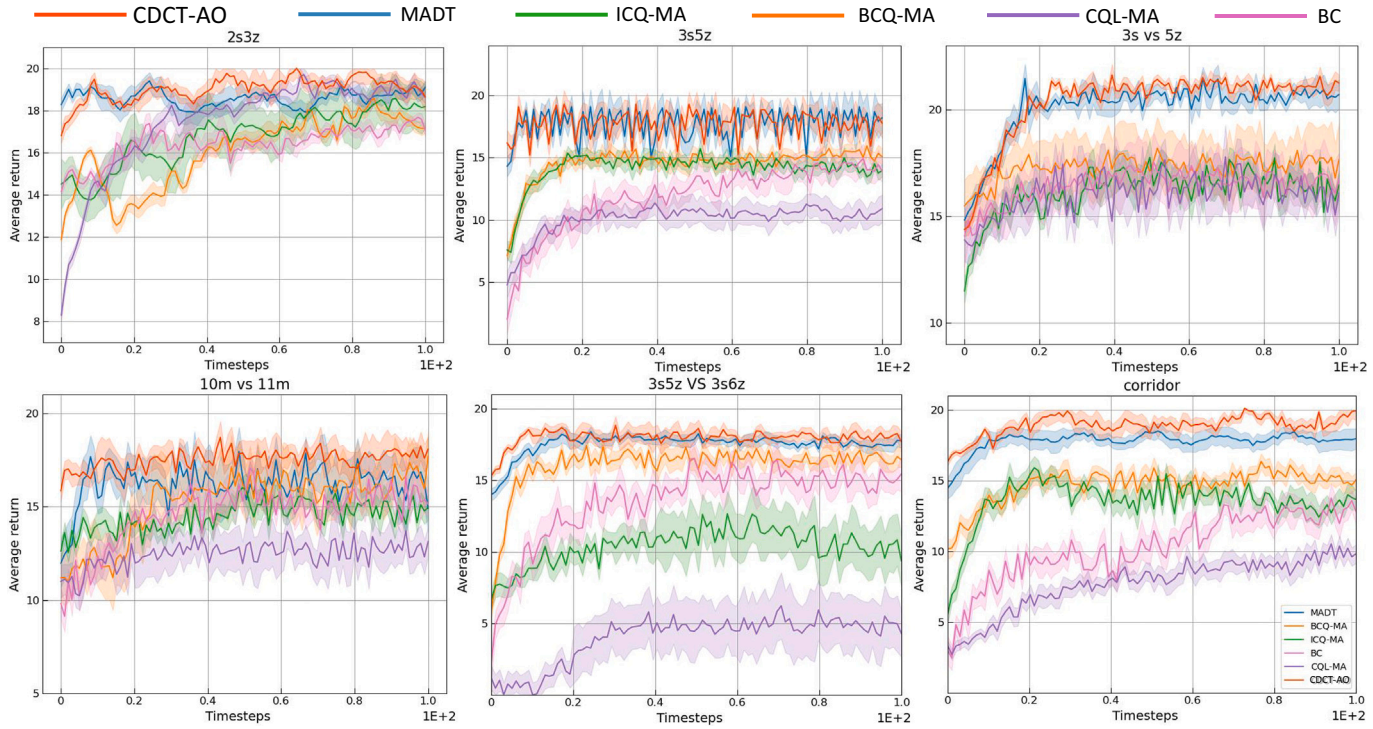


Fig. 7. Comparison of offline training performance between our CDCT-AO's MA-CCT actor and SOTA baselines, evaluating their ability to handle distributional shift and generalization ability from offline data. The `y-axis` indicates the average return, and the `x-axis` represents the progression of training epochs. All reported results are averaged over a fixed set of 5 independent runs with different random seeds. Shaded areas indicate the standard deviation across runs.

dataset coverage. In contrast, CDCT-AO continues to improve throughout training, suggesting a stronger ability to extract transferable patterns from limited demonstrations. This phenomenon is most pronounced in complex scenarios (`MMM`, `3s5z vs.3s6z`) where traditional methods suffer from catastrophic divergence when facing state distribution mismatches.

Overall, these findings validate the core innovation of CDCT-AO's MA-CCT actor, whose specialized architecture, based on causal-convolutional filters and designed to target MARL's unique temporal characteristics, significantly mitigates challenges in offline learning. By better modeling localized temporal dependencies and maintaining policy consistency, MA-CCT establishes a stronger foundation for subsequent online adaptation, effectively addressing the critical limitation: offline pre-trained policies often fail to bridge the simulation-to-reality gap during fine-tuning.

5.2.5. *Evaluation on the MAMuJoCo environment*

To further demonstrate the generalization capability of our proposed CDCT-AO beyond the SMAC benchmark, we additionally evaluate it on the Multi-Agent MuJoCo (MAMuJoCo) benchmark [3,56,57]. MAMuJoCo transforms classic single-agent MuJoCo locomotion tasks into fully cooperative multi-agent settings by decomposing a robot into multiple agents, each controlling a subset of joints. There is no competition or game-theoretic interaction among agents; instead, all agents share a common objective of maximizing locomotion performance, making MAMuJoCo a challenging yet clean testbed for cooperative continuous-control MARL.

MAMuJoCo Tasks and Evaluation Protocol. We consider three representative tasks: `2halfcheetah`, `2ant`, and `4ant`, where the prefix indicates the number of agents controlling the robot. `HalfCheetah` and `Ant` correspond to robots with distinct morphologies and dynamics, thus testing robustness across different control complexities. Following standard practice, we evaluate performance under three offline dataset qualities: **Good**, **Medium**, and **Poor**. The reported metric is the Average Episode Return, computed by averaging rewards over multiple evaluation rollouts and five independent random seeds, ensuring statistical robustness.

Baselines. We compare CDCT-AO against widely adopted offline MARL baselines for continuous control, including Behavioral Cloning (BC) [31,51], MA-TD3 + BC [58], and OMAR [59]. We do not include diffusion-based MADiff in the table, as it represents a distinct modeling paradigm and is known to achieve higher absolute returns on MAMuJoCo; instead, our goal here is to assess how CDCT-AO generalizes relative to mainstream non-diffusion offline MARL methods.

Comparison Results. Table 6 summarizes the results. Overall, CDCT-AO consistently achieves strong performance across different robots, agent counts, and dataset qualities. On `2halfcheetah`, CDCT-AO obtains the highest return among the compared baselines on the **Good** dataset and remains competitive on **Medium** and **Poor** datasets, indicating robustness to reduced data quality. On `2ant` and `4ant`, CDCT-AO significantly outperforms OMAR and matches or exceeds BC and MA-TD3 + BC in most settings, particularly under **Medium** and **Poor**

Table 6
Performance comparison on MAMuJoCo benchmark. Results are reported as average episode return (mean ± std) over five random seeds.

Task (MaMuJoCo)	Dataset	BC [31]	MA-TD3 + BC [58]	OMAR [59]	CDCT-AO (Ours)
2halfcheetah	Good	6846 ± 574	7025 ± 439	1434 ± 1903	**7314 ± 682**
	Medium	1627 ± 187	2561 ± 82	1892 ± 220	2336 ± 165
	Poor	465 ± 59	736 ± 72	384 ± 420	**684 ± 37**
2ant	Good	2697 ± 267	2922 ± 194	464 ± 469	2842 ± 364
	Medium	1145 ± 126	744 ± 283	799 ± 186	**1235 ± 156**
	Poor	954 ± 80	1256 ± 122	857 ± 73	1105 ± 70
4ant	Good	2802 ± 133	2628 ± 971	344 ± 631	2740 ± 235
	Medium	1617 ± 153	1843 ± 494	929 ± 349	**1532 ± 156**
	Poor	1033 ± 122	1075 ± 96	518 ± 112	**1095 ± 65**

datasets where distributional shift is more severe. These results highlight several important observations. First, CDCT-AO generalizes effectively from discrete, partially observable SMAC environments to continuous-control, fully cooperative MAMuJoCo tasks, demonstrating architectural versatility. Second, the performance gap between CDCT-AO and value-based baselines (e.g., MA-TD3 + BC, OMAR) becomes more pronounced as dataset quality degrades, suggesting that the sequence-modeling and offline-to-online design of CDCT-AO is more robust to limited or sub-optimal offline data. Third, despite not being specifically tailored for continuous control, CDCT-AO maintains stable returns with relatively small variance across seeds, indicating reliable multi-agent coordination learning.

In summary, the MAMuJoCo experiment in this part suggests that CDCT-AO can generalize across different state-action spaces, agent granularities, and data qualities, supporting its applicability as a general offline-to-online MARL framework.

5.2.6. Scalability evaluation on large-scale SMAC scenarios

To further examine the scalability of the proposed CDCT-AO framework beyond small-scale agent teams, we conduct additional experiments on large-scale SMAC scenarios involving more than 20 agents. Specifically, we evaluate performance on the `25m` and `27m vs.30m` maps, which require coordinated control of 25 and up to 30 units, respectively. These tasks substantially increase the complexity of joint action spaces, coordination demands, and credit assignment difficulty, providing a meaningful test of scalability.

The experimental protocol follows the same offline-to-online training pipeline as in our main SMAC experiments. We compare CDCT-AO against representative offline and offline-to-online MARL baselines, including MADT, BCQ-MA, ICQ-MA, BC, and CQL-MA. Shaded regions in the learning curves indicate performance variability across runs.

Fig. 8 presents the learning curves on large-scale scenarios. Fig. 8(a) shows results on `25m`, while Fig. 8(b) corresponds to `27m vs.30m`. Across both scenarios, CDCT-AO consistently achieves higher returns and faster convergence compared to all baselines. Notably, the performance gap becomes more pronounced as training progresses, indicating superior scalability under increasing coordination complexity.

Despite the substantially larger number of agents, CDCT-AO maintains stable learning dynamics without performance collapse or excessive variance. This suggests that the MA-CDCT actor architecture and the offline-to-online optimization pipeline scale effectively with team size. In contrast, several baselines exhibit slower convergence and lower asymptotic performance, highlighting their difficulty in handling large joint action spaces.

These results demonstrate that CDCT-AO is not restricted to small-team SMAC settings and can scale to scenarios with 25–30 agents. The strong performance on large-scale maps provides empirical evidence supporting the scalability of the proposed framework, thereby strengthening its potential applicability to larger cooperative multi-agent systems.

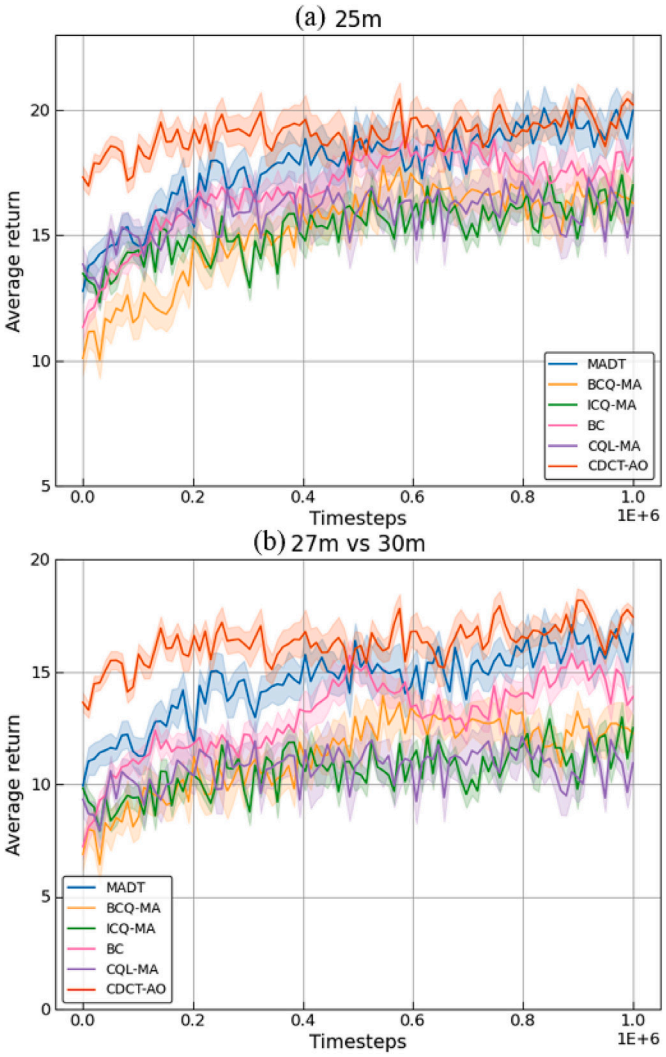


Fig. 8. Scalability evaluation on large-scale SMAC scenarios. (a) `25m`. (b) `27m vs.30m`. Curves report average episode return, with shaded regions indicating variability across runs.

5.3. Ablated studies

In this section, we validate the effectiveness of each module in CDCT-AO through four ablation experiments. Each experiment disables one of the proposed modules to form a corresponding variant model for comparison.

5.3.1. Ablation of MA-CCT: causal-convolution vs. transformer MHSA:

In this ablation experiment, we investigate the performance of the proposed Multi-Agent Causal-Convolutional Transformer (MA-CCT) by comparing it to the original Transformer architecture that uses MHSA for action sequence prediction. The MA-CCT module replaces the MHSA with causal convolution filters in the CDCT-AO framework to better capture localized temporal dependencies and enhance generalization, especially for handling OOD states in offline MARL tasks.

As demonstrated in Fig. 9, we evaluated the ablated variant, '*CDCT-AO with self-attention*', alongside our default '*CDCT-AO with causal-convolutional*' filtering on three maps: '3s5z vs.3s6z', 'MMM', and 'corridor'. From this figure, the MA-CCT consistently outperforms the original Transformer with MHSA across multiple challenging SMAC maps. On maps such as '3s5z vs.3s6z' and 'corridor', the MA-CCT architecture demonstrates superior performance in both initial return and convergence speed, showcasing its ability to learn and adapt more effectively in offline-to-online training settings.

The key reason for the improved performance of MA-CCT is its ability to capture local temporal dependencies between adjacent timesteps. While the MHSA mechanism in the original Transformer struggles to model such localized associations, the causal-convolution filters in MA-CCT specifically address this limitation, allowing MA-CCT to more accurately represent the Markovian properties intrinsic to the multi-agent settings. Additionally, MA-CCT's focus on localized temporal structures makes it highly efficient in modeling the dynamics of POMDPs, which are common in MARL tasks. This results in a more robust and generalizable policy that can handle previously unseen states during fine-tuning, an issue that often leads to suboptimal performance in traditional Transformer-based architectures.

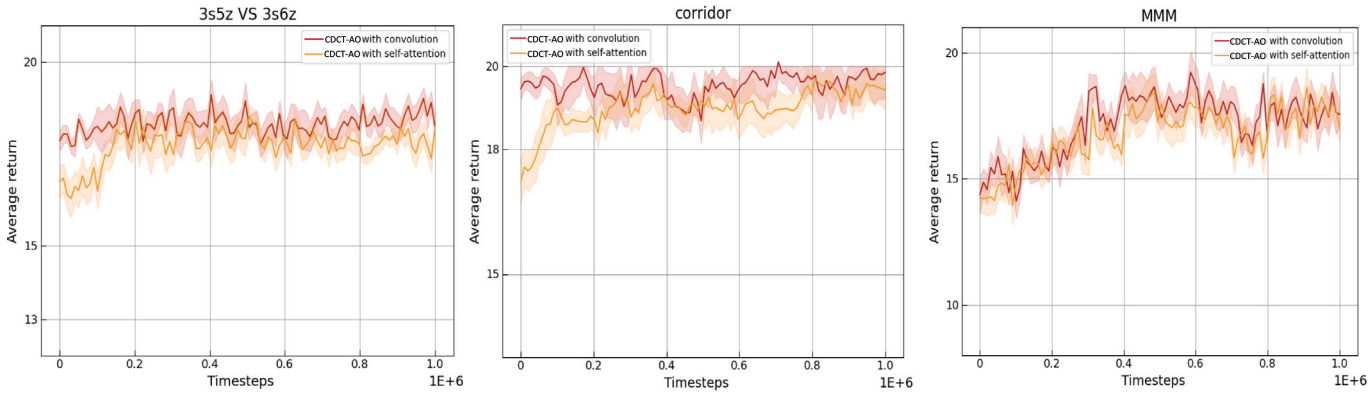


Fig. 9. Performance comparison of the online Fine-Tuning Procedure between our proposed MA-CCT and the original Decision Transformer with MHSA on three Offline-to-Online MARL Tasks, '3s5z vs.3s6z', 'MMM', and 'corridor'. All results are reported as the mean ± standard deviation over 5 independent runs with different random seeds.

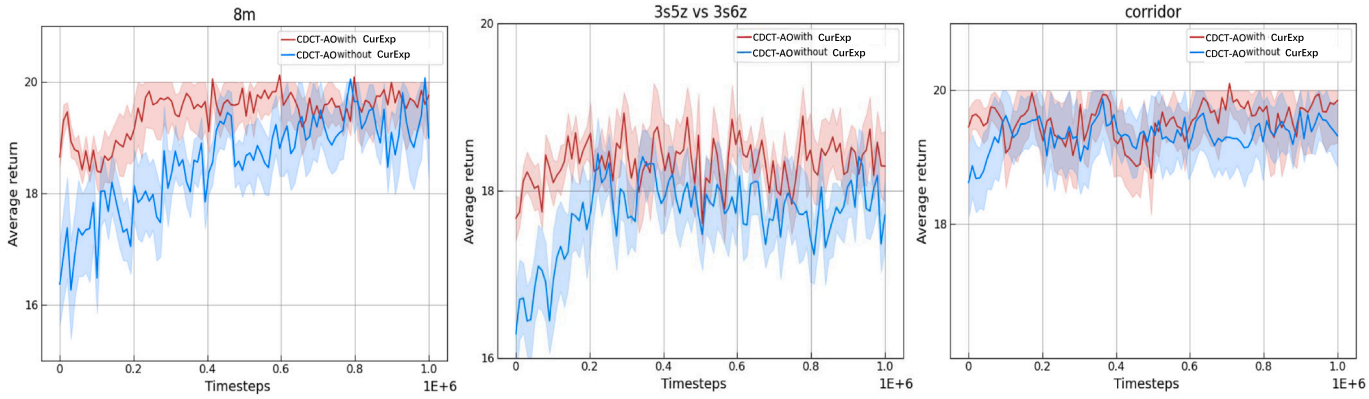


Fig. 10. Performance comparison of the online Fine-Tuning Procedure between our proposed Curiosity-Driven exploration (CurExp) and the approach without CurExp on three Offline-to-Online MARL Tasks, '8m', '3s5z vs.3s6z', and 'corridor'. All reported results are averaged over 5 independent runs with different random seeds. Shaded areas indicate the standard deviation across runs.

5.3.2. Curiosity-driven exploration vs. without curiosity-driven exploration

Here, we analyze the impact of the CurExp mechanism on the performance of the CDCT-AO framework across three challenging maps: '8m', '3s5z vs.3s6z', and 'corridor'. CurExp is designed to enhance exploration efficiency in multi-agent reinforcement learning (MARL) by guiding agents toward underexplored states, thereby improving generalization in OOD scenarios. When CurExp is active, the model uses prediction errors from the next-state and action transitions to motivate exploration, helping to bridge the gap between offline pre-training and online fine-tuning.

In this ablation, the key difference between the two configurations is whether CurExp is enabled or not, as determined by comparing the performance of average returns, as shown in Fig. 10. In the figures, CDCT-AO that enables CurExp significantly improves the performance of CDCT-AO without CurExp. Specifically, in the '8m' map, CDCT-AO with CurExp (red curve) consistently shows higher average returns compared to the version without CurExp (blue curve). The same trend is observed in the more complex '3s5z vs.3s6z' and 'corridor' maps, where CurExp-enhanced CDCT-AO outperforms the non-curiosity-driven version of CDCT-AO in both initial return and convergence speed. CurExp boosts exploration efficiency, enabling the agents to cover a broader range of states during the fine-tuning phase. This expanded exploration allows CDCT-AO to adapt better to unseen states, which is critical for tackling OOD challenges in MARL tasks.

From this, we can see that CurExp significantly contributes to CDCT-AO's ability to explore more effectively and generalize better in real-world environments. The improved performance across all three maps indicates that CurExp is an essential mechanism for overcoming exploration inefficiencies in offline-to-online training, ultimately enhancing the robustness of the model.

5.3.3. Ablation of our self-adaptive MSAL with other learning rate schedulers

In this ablation, we analyze the impact of three different variants on the performance of CDCT-AO during the online fine-tuning procedure, as depicted in Fig. 11. The variants include the removal of MSAL, as well as the use of two different LR schedulers: *StepLR*[12] and *CosineAnnealingLR*.[13]. The first variant is to compare the performance of CDCT-AO with and without the MSAL module while using a fixed learning rate, as shown by the red curve and the green curve. The pink curves denote the StepLR-based CDCT-AO variant, which applies the StepLR scheduler, a step decay to the learning rate after a fixed number of timesteps. The purple curves denote the CosineAnnealingLR-based CDCT-AO variant, which applies the CosineAnnealingLR scheduler, which adjusts the learning rate following a cosine decay pattern, where the learning rate gradually decreases over time and can help in finer exploration during later stages of training. These experiments were conducted on three maps: 'corridor', '3s vs.5z', and '3s5z vs.3s6z'.

[12] StepLR: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.StepLR.html

[13] CosineAnnealingLR: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html

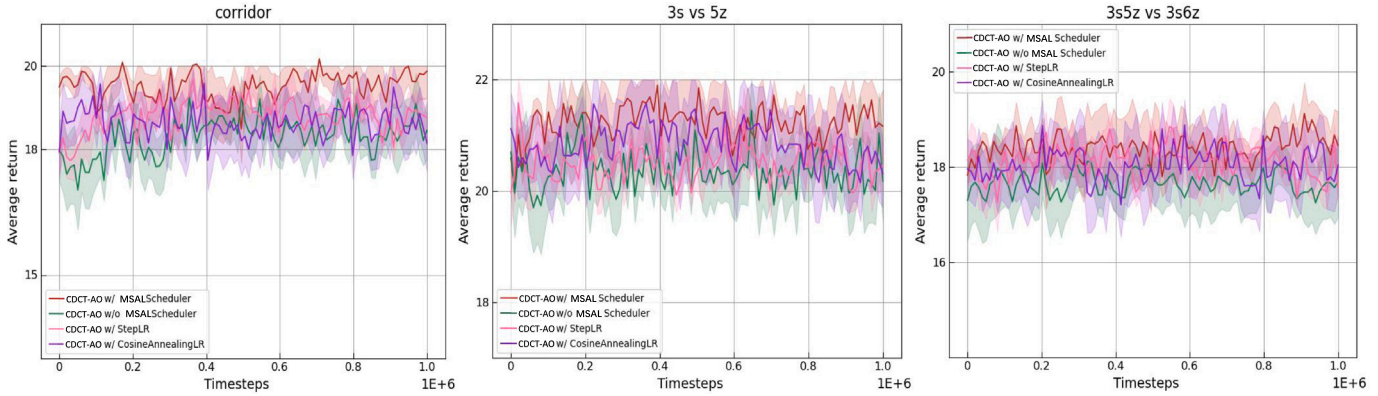


Fig. 11. Performance comparison of the online Fine-Tuning Procedure between the CDCT-AO variants that adopt different Learning-Rate schedulers on 'corridor', '3s vs.5z', and '3s5z vs.3s6z' to validate the effectiveness of the MSAL Module. All reported results are averaged over a fixed set of 5 independent runs with different random seeds. Shaded areas indicate the standard deviation across runs.

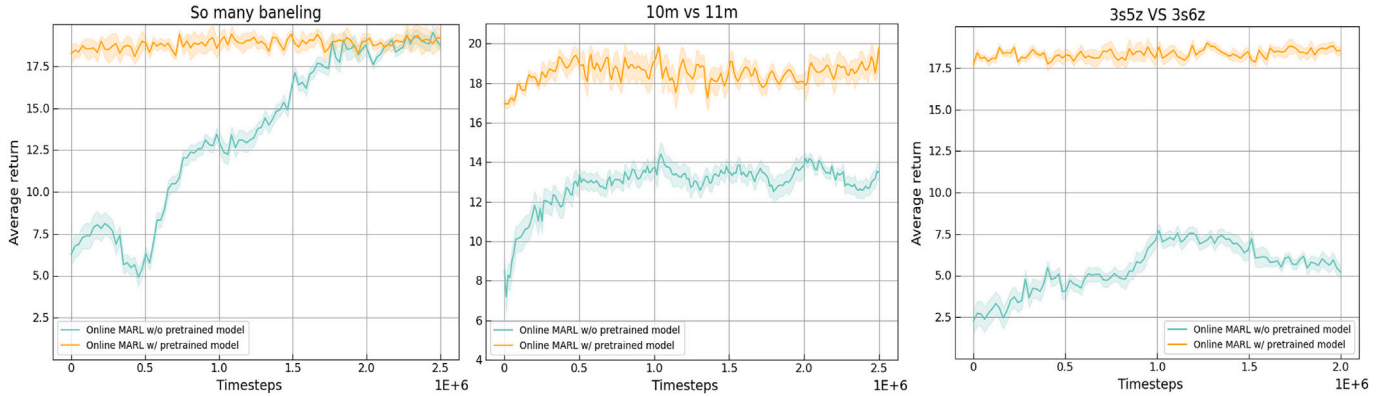


Fig. 12. Performance comparison on mean episode return of CDCT-AO variants with pre-training and pure online training across three maps: *10m vs.11m*, *so_many_baneling*, and *3s5z vs.3s6z*. The results are averaged over 5 independent runs with different random seeds, and reported as the mean ± standard deviation.

From these results, the CDCT-AO variant without MSAL struggles to achieve higher performance compared to CDCT-AO with MSAL, indicating that MSAL is critical for better exploration and generalization during online fine-tuning. The addition of MSAL improves the average return across all the '3s5z vs.3s6z', 'corridor', and '3s vs.5z' maps, as shown by the red and green curves. This finding evaluates the effect of MSAL on the learning dynamics of the model and demonstrates that CDCT-AO with the MSAL module consistently outperforms the version without it across all maps. As for the two variants of StepLR and CosineAnnealingLR, the influence of these different LR schedulers on CDCT-AO's performance is further explored. CDCT-AO with StepLR generally shows more stable convergence over time compared to a fixed learning rate, but it still lags slightly behind the CosineAnnealingLR-based variant. Meanwhile, CDCT-AO with CosineAnnealingLR exhibits the best overall performance, with higher average returns and faster convergence in comparison to StepLR and the fixed learning rate variants across all three maps.

To summarize, this ablation clearly demonstrates the importance of both the MSAL module and an appropriate learning rate scheduler in enhancing the performance of the CDCT-AO framework. Removing MSAL results in a noticeable drop in performance, highlighting its crucial role in CDCT-AO's performance. This suggests that dynamic learning rate adjustments play a key role in improving training efficiency and final performance in such complex offline-to-online MARL tasks.

5.3.4. *Offline pre-training of CDCT-AO vs. pure online training variant*

In this part, we compare the performance of pure online learning and offline pretraining followed by online fine-tuning, as depicted by blue and yellow curves in Fig. 12. We conduct this comparison on three different maps, as shown in the figure, which include the 'So_many_baneing', '10m vs.11m', and '3s5z vs.3s6z' maps. For *Pure Online Learning*, under this setting, the CDCT-AO model is trained by interacting with the environment without any offline data. It can be observed that the model's performance is initially poor, especially on the 'So_many_baneing' and '10m vs.11m' maps, where the average return takes a long time to improve significantly. In the initial phase, due to the lack of prior knowledge about the environment, the model has to rely on random exploration to learn effective strategies, resulting in low returns.

For the CDCT-AO model that undergoes pre-training, i.e., with offline MARL training, it starts the online fine-tuning phase with some initial strategy knowledge, significantly reducing the training time. On all three maps, the pretrained CDCT-AO shows faster convergence. Especially on the 'So_many_baneing' and '10m vs.11m' maps, the pretrained CDCT-AO achieves higher average returns early on, indicating that pretraining allows CDCT-AO to adapt more quickly and obtain better strategies. On the basis of these findings, we conclude that pretraining helps the model learn effective action-state mappings during the offline phase. The offline learning phase incorporates rich interaction data, enabling the model to accumulate diverse experiences. As a result, during the online fine-tuning stage, the model can optimize its strategy more efficiently and adapt rapidly, particularly in more complex scenarios.

This experiment demonstrates that offline pretraining provides significant benefits for improving online learning performance. By offering our CDCT-AO more comprehensive initial learning, pretraining allows it to adapt faster to new tasks and environments in the online phase, especially in complex and challenging tasks. This strategy is particularly

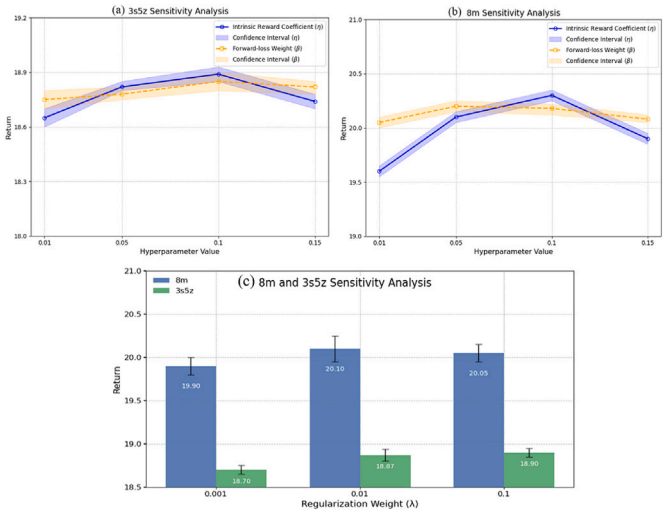


Fig. 13. Sensitivity analysis of CurExp hyperparameters. (a) Impact of intrinsic reward coefficient η and forward-loss weight β on the `3s5z` scenario. (b) Impact of intrinsic reward coefficient η and forward-loss weight β on the `8m` scenario. (c) Sensitivity analysis of the entropy bonus coefficient across representative SMAC tasks.

important in multi-agent reinforcement learning (MARL), as it effectively reduces training time and improves the policy's decision-making quality.

5.3.5. Sensitivity analysis of CurExp hyperparameters

To further examine the robustness of the proposed Curiosity-driven Exploration (CurExp) mechanism, we conduct a sensitivity analysis on its key hyperparameters, including the intrinsic reward coefficient η, the forward-loss weight β, and the entropy bonus coefficient. All other hyperparameters are fixed to the default CDCT-AO configuration, and evaluations are carried out on representative SMAC scenarios. The results are summarized in Fig. 13.

Intrinsic reward coefficient η. Fig. 13(a) and (b) illustrate the influence of the intrinsic reward coefficient $\eta \in \{0.01, 0.05, 0.1, 0.15\}$ on the average episode return. Across different maps, the performance exhibits a consistent unimodal trend. When η is too small, the intrinsic reward signal is weak, limiting the agents' incentive to explore states beyond the offline data distribution. Increasing η leads to improved returns, indicating more effective exploration and better adaptation to unseen states. However, excessively large η slightly degrades performance, suggesting that overly strong intrinsic rewards may interfere with task-driven optimization. Overall, moderate values of η achieve the best balance between exploration and exploitation, while maintaining stable performance with relatively small variance.

Forward-loss weight β. Fig. 13(a) and (b) also report the sensitivity to the forward-loss weight β, which controls the trade-off between forward dynamics prediction and inverse action prediction in CurExp. We observe that performance remains stable over a broad range of β values. Smaller β emphasizes the inverse model and focuses more on action-relevant representation learning, whereas larger β prioritizes modeling state-transition dynamics. Intermediate settings consistently achieve strong performance, indicating that CurExp benefits from jointly learning dynamics awareness and control-relevant features, rather than relying exclusively on either component.

Entropy bonus coefficient. Fig. 13(c) presents the effect of varying the entropy bonus coefficient. Very small entropy bonuses tend to reduce exploration and lead to slightly lower returns, while excessively large entropy bonuses introduce additional stochasticity and slow convergence. A moderate entropy coefficient provides a favorable trade-off, yielding stable learning dynamics and competitive returns across tasks. Notably, performance differences across reasonable entropy settings remain limited, suggesting that the overall framework is not overly sensitive to this parameter.

In summary, the sensitivity analysis demonstrates that CDCT-AO with CurExp maintains robust performance across a wide range of hyperparameter settings. The results indicate that the proposed intrinsic motivation mechanism does not depend on narrowly tuned coefficients, and that its performance gains persist under moderate variations of η, β, and entropy regularization.

5.3.6. Ablation on the number of MA-CCT layers

In this section, we investigate the impact of the network depth of the proposed MA-CCT actor by varying the number of stacked MA-CCT blocks. While deeper architectures generally increase model capacity, they may also introduce optimization difficulties and overfitting in offline-to-online MARL. Therefore, it is important to examine how the depth of MA-CCT affects performance in practice.

Experimental Setup. We evaluate MA-CCT architectures with different numbers of layers, ranging from 1 to 5 stacked MA-CCT blocks. All other hyperparameters are kept identical to the default CDCT-AO configuration. The ablation is conducted on two representative and challenging SMAC scenarios: `3s5z vs.3s6z` and `10m vs. 11m`. For each configuration, we report results for both the offline pre-training phase and the online fine-tuning phase. Following our main experiments, the evaluation metric is the Mean of Top-20 Highest Returns, which is computed by averaging the top-20 highest episode returns across multiple evaluation runs. This metric reflects the achievable performance ceiling and provides a stable measure of high-quality policy behavior.

Results & Discussion. Fig. 14 summarizes the results. Fig. 14(a) corresponds to the `3s5z vs.3s6z` scenario, while Fig. 14(b) reports results on `10m vs. 11m`. Several consistent trends can be observed across both tasks and training stages.

First, MA-CCT with 2 layers achieves the best overall performance in both offline and online settings. In particular, the 2-layer configuration yields the highest mean top-20 returns after online fine-tuning, indicating superior policy adaptation and performance ceiling. Second, shallow architectures with only 1 layer underperform compared to the 2-layer variant. Third, increasing the depth beyond 2 layers does not lead to further performance gains. On the contrary, models with 3, 4, or 5 layers exhibit a gradual degradation in both offline and online performance. This phenomenon is consistently observed across both scenarios. We attribute this behavior to the fact that deeper MA-CCT stacks may introduce over-smoothing of temporal features or increased optimization difficulty, especially under offline data constraints, which limit effective credit assignment across layers.

Overall, this ablation demonstrates that the performance of CDCT-AO is sensitive to the depth of the MA-CCT actor, but does not benefit from excessive depth. A 2-layer MA-CCT strikes an effective balance between expressive power and training stability, enabling robust offline learning and efficient online adaptation. These results justify our default architectural choice and highlight that the proposed causal-convolutional design is most effective when applied with moderate depth rather than deep stacking.

5.3.7. Cross-dataset evaluation with imperfect demonstrations

To further evaluate robustness under distribution shift in offline data, we conduct a cross-dataset validation experiment that explicitly separates the data distributions used for **offline pre-training** and **fine-tuning**. The key idea is to pre-train on a high-quality expert dataset (MAPPO demonstrations) and then adapt the policy using an **imperfect/noisy** demonstration set that follows a different and degraded behavioral distribution, thereby testing whether the learned representation and optimization pipeline can remain stable under imperfect supervision.

We consider two typical SMAC scenarios, `25m` and `27m vs.30m`. We first perform offline pre-training on expert trajectories collected by MAPPO (denoted as $\mathcal{D}_{\text{expert}}$), following the same settings as our main

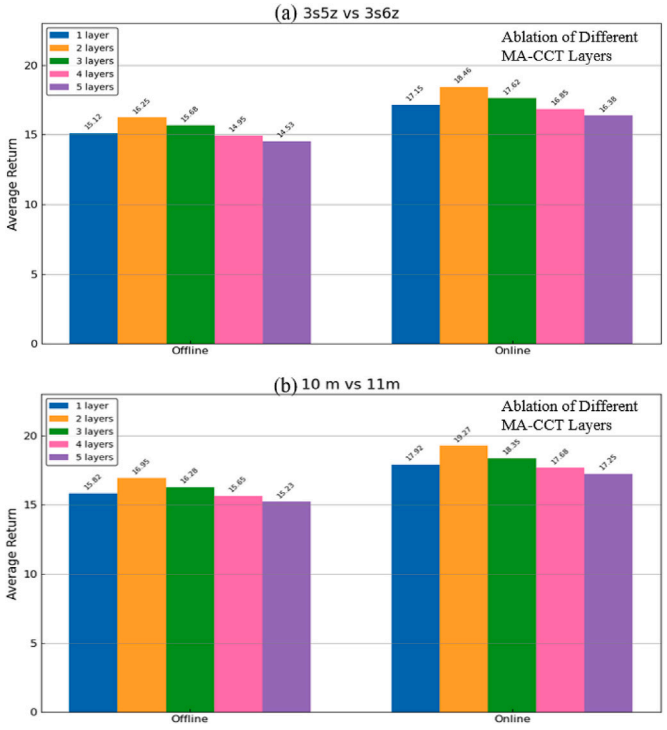


Fig. 14. Ablation study on the number of MA-CCT layers. (a) Results on the `3s5z vs.3s6z` scenario. (b) Results on the `10m vs. 11m` scenario. Performance is measured by the mean of the Top-20 highest returns for both offline pre-training and online fine-tuning.

experiments. Next, before online interaction, we introduce an additional **offline adaptation** stage using a corrupted dataset $\mathcal{D}_{\text{noisy}}$ constructed from $\mathcal{D}_{\text{expert}}$ via controlled perturbations. Specifically, for each transition $(s_t, o_t, a_t, r_t, s_{t+1})$, we apply the following noise model: (1) **Action corruption:** with probability $p_a = 0.3$, the executed action is replaced by a random legal action under the action mask, simulating suboptimal or erroneous decisions;(2) **Reward perturbation:** we add zero-mean noise $r'_t = r_t + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma_r^2)$ and $\sigma_r = 0.5$, modeling logging uncertainty and imperfect reward annotation; (3) **Trajectory degradation:** we additionally inject partial trajectory truncation (randomly cutting the last 10%–20% steps) to mimic incomplete demonstrations. This yields a shifted and degraded dataset distribution while preserving the original environment dynamics. We then fine-tune the pre-trained model on $\mathcal{D}_{\text{noisy}}$ for a fixed number of epochs using the same training pipeline, and finally perform standard online fine-tuning as in the main setting.

We report online learning curves of average episode return for: (i) **MADT**, (ii) **CDCT-AO** (standard: expert pre-training + online fine-tuning), and (iii) **CDCT-AO + noisy-demo adaptation** (expert pre-training + offline adaptation on $\mathcal{D}_{\text{noisy}}$ + online fine-tuning). Results are shown in Fig. 15, where shaded regions indicate variability across runs.

As shown in Fig. 15(a) & (b), CDCT-AO remains robust under cross-dataset shift. After adapting on imperfect demonstrations, the resulting performance curve (**CDCT-AO with fine-tune to imperfect data**) stays close to the standard CDCT-AO curve and consistently outperforms MADT in both scenarios. Notably, although noisy-demo adaptation introduces a weaker supervision signal, CDCT-AO does not collapse or exhibit unstable oscillations; instead, it preserves competitive returns and converges to a similar performance regime. This indicates that the proposed framework is not overly dependent on a narrowly curated expert dataset and can tolerate substantial imperfections in the fine-tuning distribution.

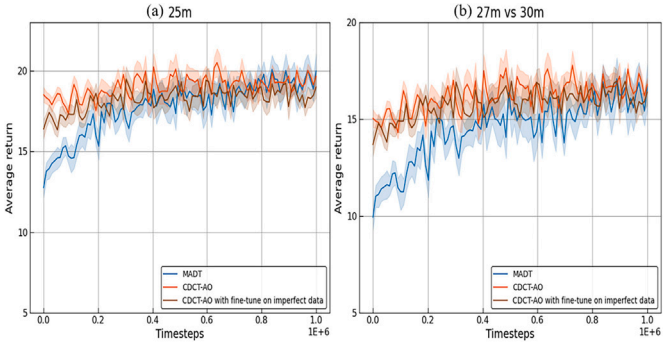


Fig. 15. Cross-dataset evaluation under imperfect demonstrations. (a) `25m`. (b) `27m vs.30m`. Policies are pre-trained on MAPPO expert demonstrations and then adapted using a noisy demonstration dataset before online fine-tuning.

Overall, this cross-dataset evaluation provides direct evidence that CDCT-AO can generalize across offline data distributions and maintain stable learning under imperfect demonstrations.

6. Case study

6.1. Real-time decision-making evaluations

To explicitly demonstrate the superiority of our CDCT-AO compared to other offline-to-online baselines such as MADT, we conducted a series of real-time decision-making evaluations in MARL to help readers better understand the effectiveness of CDCT-AO. As depicted in Figs. 16 and 17, we present two real-time rollouts on the '*corridor*' and '*3s5z vs.3s6z*' super-hard maps[14] to visualize the superiority of our CDCT-AO framework over the offline-pretrained MADT baseline. For each map, we randomly select one successful battle executed by **CDCT-AO** (left, six snapshots equally spaced from t=0 to t=T) and one failed battle executed by **MADT** (right, same temporal slicing). Below we analyze the decision dynamics at representative timesteps.

For the case of 'corridor' (Fig. 16). (1) `t=0` → `t=1/5 T`. CDCT-AO deploys '*Zealots*' to immediately anchor the '*choke*', forming a convex frontline that denies flanking angles. In contrast, MADT advances uniformly into the corridor mouth, leaving side gaps that invite wrap-around by '*Zerglings*'. (2) `t=2/5 T`. Guided by MA-CCT's local temporal modeling, CDCT-AO executes short *stutter-steps* and micro-retreats while keeping a tight body-block, so the first '*Zergling*' wave collides with a compact wall and trades unfavorably. MADT shows *timing drift*: its front '*Zealots*' desynchronize by a few frames, creating a seam; the enemy begins to thread through. (3) `t=3/5 T`. Our curiosity-driven exploration, CurExp, has during fine-tuning exposed the policy to high-uncertainty surround attempts; CDCT-AO therefore triggers a side-step collapse (two '*Zealots*' pinch inward) to re-seal the opening. MADT fails to re-form; its formation fractures into two mini-lines, and '*Zerglings*' split to encircle. (4) `t=4/5 T` → `T`. With MSAL's stabilized learning-rate schedule, CDCT-AO maintains consistent rhythm in hit-and-hold micro, thinning '*Zerglings*' before any full surround materializes; casualty snowball then flips decisively. MADT is fully wrapped; once the rear Zealots are isolated, the remaining front cannot hold the surface, leading to collapse near t=T.

[14] **Unit compositions and races.** In the '*3s5z vs.3s6z*' scenario, both our team and the opponent are Protoss units: a flexible mix of ranged **Stalkers** and melee **Zealots**. Our team has the composition of 3s + 5z against the opponent's 3s + 6z, which is asymmetric and heterogeneous. In the '*corridor*' scenario, our team consists of 6 **Zealots**, while the opponent has 24 **Zerglings** (Zerg). The map features a narrow corridor (choke) in the center, requiring strict utilization of terrain to avoid being surrounded.

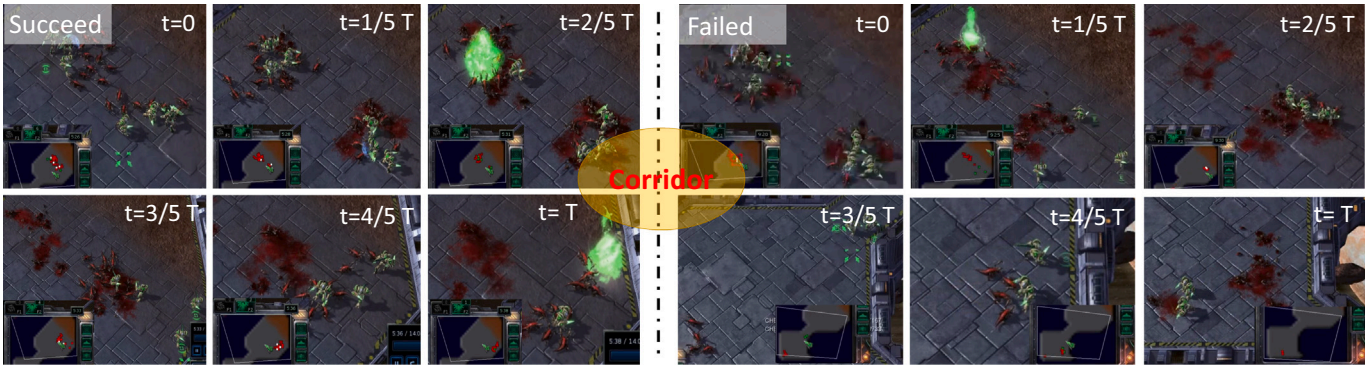


Fig. 16. Real-time battle scenarios in the '*corridor*' map: CDCT-AO (left) successfully defends the units at '*choke*', while MADT (right) fails to maintain formation control. **Specific tactical Differences to Observe:** (1) **Choke-point Control:** CDCT-AO forms a convex frontline at the corridor entrance to deny flanking, while MADT advances uniformly, leaving gaps. (2) **Formation Cohesion:** Under pressure, CDCT-AO maintains a tight body-block with synchronized micro-retreats; MADT's formation desynchronizes, allowing enemy penetration. (3) **Adaptive Adjustment:** CDCT-AO executes pinch maneuvers (e.g., side-step collapse) to seal openings; MADT fails to reform its fractured line.

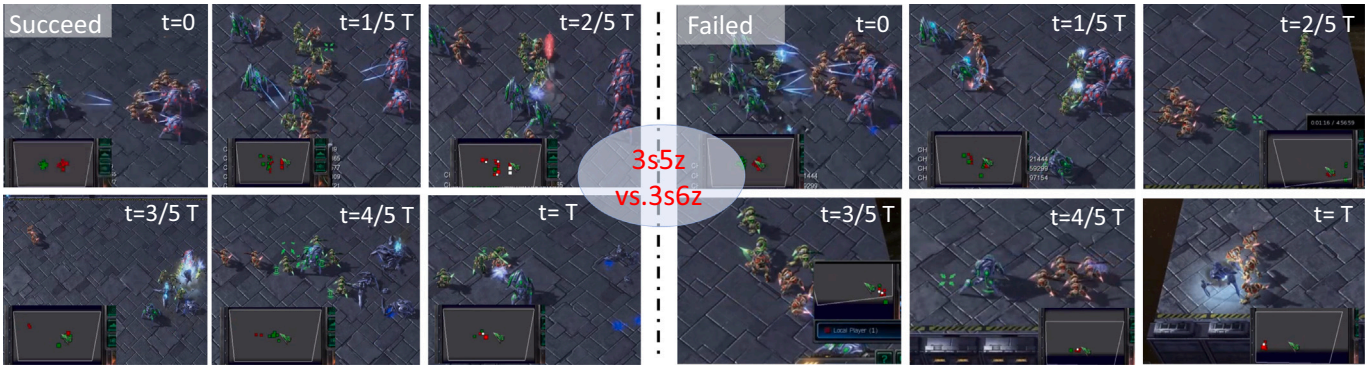


Fig. 17. Real-time battle scenarios in the '*3s5z vs.3s6z*' map: CDCT-AO (left) successfully executes tactical kiting, while MADT (right) struggles with formation coherence. **Specific tactical Differences to Observe:** (1) **Initial Positioning:** CDCT-AO optimally spaces Zealots (screen) and Stalkers (diagonal firing lane); MADT clusters Stalkers, limiting kiting. (2) **Focus Fire:** CDCT-AO concentrates fire to eliminate key targets; MADT disperses damage. (3) **Counter-Envelopment:** When wrapped, CDCT-AO coordinates Stalker retreat with Zealot body-blocks; MADT's Stalkers get caught. (4) **Late-Fight Stability:** CDCT-AO shifts focus fire stably; MADT exhibits erratic over-commit/retreat cycles.

For the case of '3s5z vs.3s6z' (Fig. 17). (1) `t=0`. CDCT-AO opens with *role-specialized spacing*: '*Zealots*' angle to screen, while '*Stalkers*' keep a diagonal lane for *kiting and focus fire*. MADT keeps '*Stalkers*' too close to the Zealot line, limiting their maneuvering room. (2) `t=1/5 T` → `2/5 T`. CDCT-AO's MA-CCT token mixer captures *short-horizon dependencies* (two–three-step micro cycles), producing crisp target-switches: Stalkers delete an overextended enemy '*Zealot*' in two volleys while our '*Zealots*' avoid over-chasing. MADT splits fire among multiple targets; damage disperses and fails to secure the first pick. (3) `t=3/5 T`. When the enemy attempts a mid-fight *wrap and collapse* (leveraging its extra Zealot), CurExp-trained CDCT-AO anticipates the OOD pattern and issues a micro-retreat for '*Stalkers*' along their reserved lane, simultaneously *body-blocking* with two '*Zealots*' to buy cooldown cycles, this preserves ranged DPS. MADT's '*Stalkers*', having no clear egress corridor, get clipped by enemy '*Zealots*'; once a Stalker falls, the fight irreversibly tilts. (4) `t=4/5 T` → `T`. With MSAL entering a *fine-tune phase*, CDCT-AO lowers its step size and avoids late-fight oscillations: focus fire shifts from red-HP '*Zealots*' to exposed '*Stalkers*', producing a clean 2–3 unit lead at t=T. The multi-agents controlled by MADT exhibit short over-commits then panicked retreats, which waste cooldowns and cede unit trades, ultimately leading to mission failure.

Quantitative analysis via remaining unit dynamics. To complement the qualitative rollout analysis and reduce subjectivity, we further quantify the battle evolution by tracking the number of surviving units over time. Table 7 reports the remaining allied and enemy unit counts at the same six timesteps used in Figs. 16 and 17.

Table 7
Remaining unit counts over time in case-study rollouts. For each map, we report the number of surviving allied and enemy units at six evenly spaced timesteps (from $t = 0$ to $t = T$).

Map	Method	Side	t=0	t=1/5T	t=2/5T	t=3/5T	t=4/5T	t=T
Corridor	CDCT-AO	Allies (6Z)	6	6	6	5	4	3
		Enemies (24L)	24	20	15	9	4	0
	MADT	Allies (6Z)	6	5	3	1	0	0
		Enemies (24L)	24	22	19	15	11	9
3s5z vs. 3s6z	CDCT-AO	Allies (3S + 5Z)	8	8	7	6	5	4
		Enemies (3S + 6Z)	9	8	6	4	2	1
	MADT	Allies (3S + 5Z)	8	7	5	3	1	0
		Enemies (3S + 6Z)	9	8	7	6	4	3

In the *corridor* scenario, CDCT-AO preserves all six Zealots until t=2/5T, while reducing the enemy Zerglings from 24 to 15, indicating a favorable early trade ratio enabled by tight choke control. In contrast, MADT begins losing frontline units as early as t=1/5T, and by t=3/5T has already lost five out of six Zealots, despite the enemy still retaining more than half of its force. This divergence quantitatively reflects the formation leakage and delayed reaction observed in the visual rollouts. A similar scenario appears in *3s5z vs. 3s6z*. CDCT-AO maintains a positive unit differential from mid-fight onward (t≥2/5T), steadily converting positional advantages into unit-level gains. MADT, however, shows a

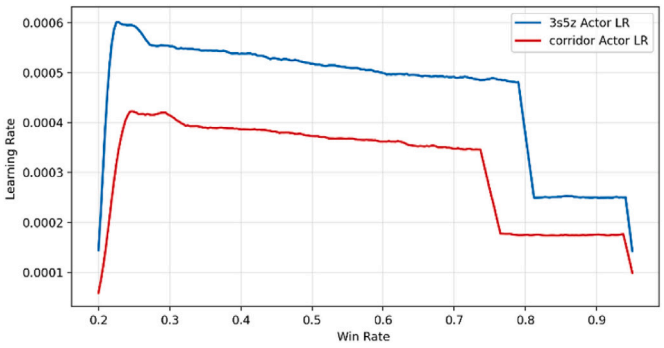


Fig. 18. Learning-rate adaptation of MSAL during online fine-tuning. The left y-axis denotes the actor learning rate, and the right y-axis denotes the win rate. MSAL dynamically adjusts the learning rate based on online performance feedback, showing exploration-oriented increases at low win rates and stabilized reductions as performance improves.

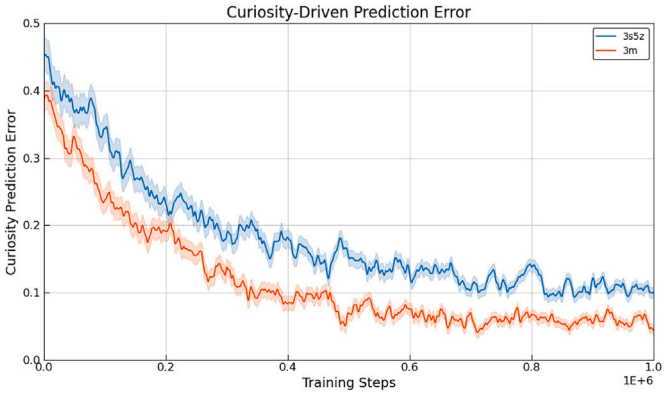


Fig. 19. Visualization of CurExp curiosity signals during online fine-tuning. The y-axis reports the curiosity prediction error, defined as the forward-model prediction loss $\|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|_2^2$, which is directly used to compute the intrinsic reward in CurExp. The x-axis denotes the number of online training steps. Higher prediction error indicates that the current transition is harder to predict (i.e., more novel or uncertain), encouraging exploration, while the decreasing trend demonstrates that the forward dynamics model progressively adapts to newly visited state-action regions during online learning.

monotonic decline in allied units while failing to decisively remove high-value enemy targets, leading to a negative unit differential that becomes irreversible after $t{=}3/5T$.

Overall, these unit-count trajectories provide a concrete, measurable proxy for tactical effectiveness, supporting our qualitative claims regarding superior formation maintenance, target prioritization, and trade efficiency of CDCT-AO during offline-to-online adaptation.

Across these cases of such two *super-hard* SMAC maps, CDCT-AO consistently converts small positional edges into winning trades, while MADT suffers from desynchronized timing, dispersed focus fire, and formation leakage. These qualitative rollouts demonstrate our quantitative gains and underscore that offline pretraining followed by online fine-tuning of CDCT-AO is a substantive advance in future MARL research.

6.2. Visualization and analysis of MSAL learning-rate dynamics

To further explore the internal dynamics of the proposed MSAL, we visualize its learning-rate adaptation during online fine-tuning in Fig. 18, together with the corresponding win-rate evolution on representative SMAC scenarios.

As shown in Fig. 18, MSAL maintains an elevated learning rate in the early fine-tuning stage, when the win rate remains low and unstable. This behavior reflects MSAL's exploration-oriented design, allowing the offline-pretrained policy to rapidly adapt to distributional shifts and explore previously unseen state-action regions in the online environment. Notably, the learning rate increases smoothly rather than abruptly, indicating stable optimization despite aggressive exploration.

As training progresses and the win rate improves, MSAL gradually reduces the learning rate. This transition corresponds to MSAL's Normal and Fine-tuning phases, where smaller step sizes help refine coordinated multi-agent behaviors and prevent oscillations around high-performing solutions.

Furthermore, Fig. 18 reveals periodic learning-rate increases at later stages of training. These spikes are triggered by MSAL's Restart mechanism and are designed to help the optimizer escape suboptimal coordination equilibria, which are common in non-convex multi-agent optimization landscapes. Importantly, these restarts do not destabilize training; instead, they are often followed by renewed performance improvements or stabilization.

Overall, the learning-rate trajectory in Fig. 18 demonstrates that MSAL achieves adaptive, non-monotonic learning-rate control aligned with policy performance. This confirms that MSAL is a key contributor to stable convergence, efficient adaptation, and robust offline-to-online transfer in CDCT-AO.

6.3. Visualizing CurExp exploration signals via curiosity prediction error

To better interpret the role of CurExp during offline-to-online fine-tuning, we visualize the evolution of the curiosity signal on representative SMAC maps (Fig. 19). In our framework, CurExp is implemented as an intrinsic curiosity module whose exploration incentive is driven by **prediction error** in a learned feature space. Concretely, given the encoded current state feature $\phi(s_t)$ and executed (joint) action a_t, the forward dynamics model $F(\cdot)$ predicts the next-state feature $\hat{\phi}(s_{t+1}) = F(\phi(s_t), a_t)$. The curiosity prediction error plotted on the **y-axis** corresponds to the forward-model regression loss (i.e., the prediction-error loss defined in our formulation):

$$\mathcal{L}_{\text{fwd}}(t) = \left\|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\right\|_2^2, \tag{24}$$

which is the quantity used to compute the intrinsic reward signal in CurExp (up to a scaling coefficient), i.e., $r_t^i \propto \mathcal{L}_{\text{fwd}}(t)$.

As shown in Fig. 19, the curiosity prediction error decreases steadily as training progresses, demonstrating a clear **explore-and-learn** pattern: CurExp initially yields large prediction errors because the online trajectory distribution deviates from the offline dataset and contains unfamiliar (OOD) transitions; as fine-tuning continues, the forward model adapts to these newly visited regions and the error reduces accordingly. We further observe map-dependent differences: the harder map (**3s5z**) maintains a consistently higher prediction error than the easier map (**3m**), indicating that exploration encounters more complex and less predictable transitions under stronger coordination requirements. This aligns with the intuition that challenging scenarios induce richer OOD dynamics, and CurExp provides a stronger intrinsic drive to cover such under-represented regions.

Overall, through this visualization, CurExp generates meaningful exploration signals during online adaptation: prediction error is high when encountering unfamiliar transitions and gradually diminishes as the policy (and curiosity models) improve coverage and predictive accuracy.

7. Limitations and discussions

Although CDCT-AO demonstrates superior performance and offline-to-online adaptation on SMAC, its transition to diverse, large-scale, and real-world intelligent systems involves several trade-offs (e.g., multi-robot teams, distributed sensing, autonomous fleets) [5–9]. Below we

discuss potential weaknesses and failure modes that arise from our designed modules, i.e., MA-CCT, CurExp, and MSAL, together with their implications for deployment.

7.1. Computational overhead and real-time constraints

While MA-CCT replaces the quadratic-cost MHSA token mixer with a linear-time causal-convolutional filter, the overall runtime can still be non-trivial in real-world systems that require low-latency inference and frequent policy updates. First, MA-CCT processes multi-modal token sequences (i.e., State, Action, and RTG) and applies modality-specific depthwise convolutions across a temporal context window; this introduces additional memory bandwidth pressure compared to simpler recurrent or MLP-based actors. Especially, practical deployment often involves on-device inference (robots, edge devices) where GPU resources and batch sizes are limited.

Moreover, CDCT-AO assumes periodic online fine-tuning with MAPPO-style optimization. In practical systems, online updates can be constrained by safety requirements, limited compute budgets, and the need to maintain uninterrupted service. Frequent fine-tuning may thus be infeasible or must be heavily rate-limited, potentially weakening the benefits of CurExp and MSAL. In the future, we will explore lightweight distillation of MA-CCT into compact actors.

7.2. Architectural comparison with MADT and resource implications

Since MADT [27] is the most closely related baseline to our approach, we provide a direct architectural comparison in terms of model size and resource footprint to clarify how much of the observed performance gain stems from architectural design rather than parameter scaling.

Table 8 reports the actor-network parameter counts of CDCT-AO and MADT under the same default configuration across representative SMAC maps. For clarity, we further convert parameter counts into approximate GPU memory usage (in MB), assuming 32-bit floating-point parameters. As shown in the table, CDCT-AO consistently uses fewer parameters and slightly less GPU memory than MADT across all evaluated scenarios.

This comparison highlights that CDCT-AO does not rely on increased model capacity to achieve superior performance. Instead, the gains primarily arise from architectural choices, namely the replacement of self-attention with causal-convolutional token mixing, which introduces a stronger inductive bias for local temporal dependencies while maintaining a compact parameterization.

7.3. Sensitivity to offline dataset quality and coverage

CDCT-AO relies on expert demonstrations for offline pre-training, and its downstream online performance is sensitive to both the quality and diversity of these trajectories. If the offline dataset contains suboptimal behaviors, narrow state coverage, or biased coordination patterns, the pre-trained MA-CDCT actor may overfit to spurious correlations (e.g., map-specific micro patterns) and perform poorly when deployed in shifted dynamics.

Meanwhile, although CurExp is designed to encourage exploration toward underrepresented states, intrinsic curiosity rewards can be insufficient to correct systematic biases if the initial policy does not work effectively. In addition, the intrinsic reward in CurExp is driven by prediction error in a learned feature space. In noisy or partially observable real-world settings, high prediction error may reflect sensor noise, or stochasticity rather than meaningful novelty, potentially leading to inefficient exploration or risk-seeking behaviors.

Furthermore, in scenarios where the offline reward signals are sparse or significantly noisy, the initial policy alignment may be insufficient, leading to a "cold-start" problem where the MSAL scheduler might trigger frequent restarts, potentially slowing down convergence during the early online stages.

Table 8
Comparison of actor-network sizes between our CDCT-AO and MADT [27]. Parameter counts are converted to approximate GPU memory usage assuming FP32 precision (4 bytes per parameter).

Method	3m	3s vs 5z	bane vs bane
CDCT-AO (~MB)	0.12	0.13	0.26
MADT (~MB)	0.15	0.15	0.27
Method	**5m vs 6m**	**corridor**	**3s5z vs 3s6z**
CDCT-AO (~MB)	0.13	0.17	0.15
MADT (~MB)	0.14	0.17	0.16

7.4. Agents scalability and large-scale coordination

Our current evaluation focuses on SMAC scenarios with moderate team sizes and fixed agent roles. Scaling CDCT-AO to significantly larger populations (dozens to hundreds of agents) presents challenges in representation, credit assignment, and training stability. Even though MA-CCT models temporal dependencies efficiently, multi-agent systems introduce combinatorial growth in joint action spaces and increasingly complex coordination structures.

At the same time, in more complex large-scale scenarios involving up to dozens of agents, (e.g., swarms of hundreds of drones or massive urban traffic control), the complexity of joint-action modeling and the intrinsic reward calculation in CurExp may scale poorly. Specifically, the forward model in CurExp predicts the feature representation of the next state based on the joint action a_t. As the number of agents increases, the action space expands exponentially, which may lead to high variance in the prediction error used for curiosity.

Furthermore, MSAL adjusts learning rates based on aggregated win-rate feedback, which may not be sufficiently informative in large-scale settings where success depends on heterogeneous sub-teams and long-horizon objectives. A single scalar performance metric can obscure agent-level failure modes (e.g., a subset of agents consistently underperforming), leading to suboptimal learning-rate oscillations.

7.5. Hyperparameter dependency in adaptive optimization

The MSAL scheduler relies on several predefined thresholds and hyperparameters, such as the win-rate threshold (wr_thres) and the epoch interval (UP_T). While these values were robust across the tested SMAC maps, their optimal settings might vary significantly in non-gaming environments, such as autonomous production logistics or disaster response. In environments where success is not a binary "win/loss" metric but a continuous performance score, defining the transition between "Normal" and "Fine-tuning" phases becomes more complex. Automating the discovery of these thresholds through meta-optimization remains an open challenge for making CDCT-AO fully "plug-and-play" for varied real-world tasks.

8. Conclusion and future work

In this work, we introduce the *Curiosity-Driven Decision Causal-Convolutional Transformer with Adaptive Optimization* (CDCT-AO) framework, a significant advancement in offline-to-online MARL. By addressing key challenges such as suboptimal expert policies in offline data and poor OOD generalization, CDCT-AO achieves superior performance in complex multi-agent environments like the SMAC. Our framework integrates three novel components: Multi-Agent Causal-Convolutional Transformer (MA-CCT), Curiosity-Driven Exploration mechanism (CurExp), and Multi-Stage Adaptive Learning-Rate Scheduler (MSAL). These innovations simultaneously enhance offline training and improve the policy adaptability during online fine-tuning, enabling better generalization, robust decision-making, and efficient exploration. Simultaneously, extensive experimental results demonstrate that CDCT-AO outperforms SOTA offline MARL methods, including TD-based approaches like BCQ-MA and IQL-MA, as well as the advanced behaviour imitation-based approaches such as MADT. Notably, CDCT-AO shows consistent improvement in both convergence

efficiency and win-rate performance across multiple challenging scenarios. This highlights the effectiveness of incorporating causal-convolution modeling, curiosity-driven exploration, and adaptive learning in enhancing the overall robustness of MARL. This work provides a valuable inspiration for future MARL that leverages offline training and online fine-tuning.

For future work, we will (i) scale CDCT-AO to larger populations via role-conditioned / hierarchical MA-CCT and factorized coordination, (ii) improve CurExp with uncertainty decomposition (epistemic vs. aleatoric) and safety-constrained exploration to avoid risk-seeking behaviors, and (iii) make deployment practical through model compression (distillation/quantization) and more stable online updates with conservative fine-tuning gates and scheduler signals beyond a single win-rate metric. Finally, we will investigate the potential of deploying CDCT-AO in practical applications, including autonomous systems and multi-robot coordination, to assess its real-world applicability and scalability.

CRediT authorship contribution statement

Bohang Shi: Writing – review & editing, Visualization, Validation, Software, Methodology, Conceptualization. **Weiqiang Jin:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Biao Zhao:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was conducted by the co-first authors, Weiqiang Jin and Bohang Shi, during their joint research at Xi'an Jiaotong University. Both authors, Weiqiang Jin and Bohang Shi, contributed equally to this work. The corresponding author is Biao Zhao. Biao Zhao is a long-term AI research collaborator with Weiqiang Jin. Lastly, the authors would like to express their appreciation for the valuable comments provided by the editors and anonymous reviewers, which improved the quality of this work.

Data availability

Full code to replicate this work are available at Github: https://github.com/albert-jin/CDCT-AO.

References

[1] Z. Ning, L. Xie, A survey on multi-agent reinforcement learning and its application, J. Autom. Intell. 3 (2) (2024) 73–91, https://doi.org/10.1016/j.jai.2024.02.003
[2] S. Gronauer, K. Diepold, Multi-agent deep reinforcement learning: a survey, Artif. Intell. Rev. 55 (2) (2022) 895–943, https://doi.org/10.1007/s10462-021-09996-w
[3] W. Jin, H. Du, B. Zhao, X. Tian, B. Shi, G. Yang, A comprehensive survey on multi-agent cooperative decision-making: Scenarios, approaches, challenges and perspectives, (2025). arXiv:2503.13415.
[4] M. Lauri, D. Hsu, J. Pajarinen, Partially observable markov decision processes in robotics: a survey, IEEE Trans. Robot. 39 (1) (2023) 21–40.
[5] A.P. Sen, M.K. Goyal, Shalini, Deploying reinforcement learning approaches for smart home Automation, in: 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), 2024, pp. 1–5, https://doi.org/10.1109/ICCCNT61001.2024.10724603
[6] W. Pinthurat, T. Surinkaew, B. Hredzak, An overview of reinforcement learning-based approaches for smart home energy management systems with energy storages, Renew. Sustain. Energy Rev. 202 (2024) 114648, https://doi.org/10.1016/j.rser.2024.114648
[7] J. Orr, A. Dutta, Multi-agent deep reinforcement learning for multi-robot applications: a survey, Sensors 23 (7) (2023), https://doi.org/10.3390/s23073625, https://www.mdpi.com/1424-8220/23/7/3625.
[8] Y. Lan, H. Gao, X. Xu, Q. Fang, Y. Zeng, Smart: sequential multiagent reinforcement learning with role assignment using transformer, IEEE Trans. Cogn. Dev. Syst. 17 (3) (2025) 615–630, https://doi.org/10.1109/TCDS.2024.3504256
[9] H.-R. Lee, T. Lee, Multi-agent reinforcement learning algorithm to solve a partially-observable multi-agent problem in disaster response, Eur. J. Oper. Res. 291 (1) (2021) 296–308, https://doi.org/10.1016/j.ejor.2020.09.018
[10] C. Shen, X. Wang, Complex task assignment of aviation emergency rescue based on multiagent reinforcement learning, in: R. Qiu, W.K.V. Chan, W. Chen, Y. Badr, C. Zhang (Eds.), City, Society, and Digital Transformation, Springer International Publishing, Cham, 2022, pp. 345–355.
[11] P. Yadav, A. Mishra, S. Kim, A comprehensive survey on multi-agent reinforcement learning for connected and automated vehicles, Sensors 23 (10) (2023), https://doi.org/10.3390/s23104710, https://www.mdpi.com/1424-8220/23/10/4710.
[12] R. Figueiredo Prudencio, M.R.O.A. Maximo, E.L. Colombini, A survey on offline reinforcement learning: taxonomy, review, and open problems, IEEE Trans. Neural Netw. Learn. Syst. 35 (8) (2024) 10237–10257, https://doi.org/10.1109/TNNLS.2023.3250269
[13] R. Zhang, J. Hou, F. Walter, S. Gu, J. Guan, F. Röhrbein, Y. Du, P. Cai, G. Chen, A. Knoll, Multi-agent reinforcement learning for autonomous driving: A survey, (2024). arXiv:2408.09675.
[14] X. Luo, H. Deng, J. Yang, Y. Shen, H. Guo, Z. Sun, M. Liu, J. Wei, S. Zhao, H2-marl: multi-agent reinforcement learning for pareto optimality in hospital capacity strain and human mobility during epidemic, Expert Syst. Appl. 291 (2025) 128432, https://doi.org/10.1016/j.eswa.2025.128432
[15] R. Zhang, Y. Sun, Z. Zhang, J. Li, X. Liu, A.H. Fan, H. Guo, P. Yan, Marl-mambacontour: Unleashing multi-agent deep reinforcement learning for active contour optimization in medical image segmentation, (2025). arXiv:2506.18679.
[16] A. Kumar, A. Zhou, G. Tucker, S. Levine, Conservative q-learning for offline reinforcement learning, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS'20, Curran Associates Inc., Red Hook, NY, USA, 2020.
[17] I. Kostrikov, A. Nair, S. Levine, Offline reinforcement learning with implicit q-learning, in: International Conference on Learning Representations (ICLR), 2022, iCLR 2022 Poster. https://openreview.net/forum?id=68n2s9ZJWF8.
[18] Y. Yang, X. Ma, C. Li, Z. Zheng, Q. Zhang, G. Huang, J. Yang, Q. Zhao, Believe what you see: implicit constraint approach for offline multi-agent reinforcement learning, in: Proceedings of the 35th International Conference on Neural Information Processing Systems, NIPS'21, Curran Associates Inc., Red Hook, NY, USA, 2021.
[19] J. Shao, Y. Qu, C. Chen, H. Zhang, X. Ji, Counterfactual conservative q learning for offline multi-agent reinforcement learning, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), Advances in Neural Information Processing Systems, vol. 36, Curran Associates, Inc., 2023, pp. 77290–77312.
[20] S. Fujimoto, D. Meger, D. Precup, Off-policy deep reinforcement learning without exploration, in: K. Chaudhuri, R. Salakhutdinov (Eds.), Proceedings of the 36th International Conference on Machine Learning, vol. 97 of Proceedings of Machine Learning Research, PMLR, 2019, pp. 2052–2062, https://proceedings.mlr.press/v97/fujimoto19a.html.
[21] R. Agarwal, D. Schuurmans, M. Norouzi, An optimistic perspective on offline reinforcement learning, in: Proceedings of the 37th International Conference on Machine Learning, ICML'20, JMLR.org, 2020.
[22] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, I. Mordatch, Decision transformer: reinforcement learning via sequence modeling, in: M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, J.W. Vaughan (Eds.), Advances in Neural Information Processing Systems, vol. 34, Curran Associates, Inc., 2021, pp. 15084–15097.
[23] J. Kim, S. Lee, W. Kim, Y. Sung, Decision convformer: local filtering in metaformer is sufficient for decision making, in: International Conference on Learning Representations (ICLR), 2024, iCLR 2024 spotlight. https://openreview.net/forum?id=af2c8EaKl8.
[24] J. Wang, P. Karanasou, P. Wei, E. Gatti, D.M. Plasencia, D. Kanoulas, Long-short decision transformer: bridging global and local dependencies for generalized decision-making, in: International Conference on Learning Representations (ICLR), 2025, iCLR 2025 Poster. https://openreview.net/forum?id=NHMuM84tRT.
[25] Z. Zhu, M. Liu, L. Mao, B. Kang, M. Xu, Y. Yu, S. Ermon, W. Zhang, Madiff: offline multi-agent learning with diffusion models, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems, vol. 37, Curran Associates, Inc., 2024, pp. 4177–4206.
[26] Q. Zheng, A. Zhang, A. Grover, Online decision transformer, in: K. Chaudhuri, S. Jegelka, Song le, C. Szepesvari, G. Niu, S. Sabato (Eds.), Proceedings of the 39th International Conference on Machine Learning, vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 27042–27059, https://proceedings.mlr.press/v162/zheng22c.html.
[27] L. Meng, M. Wen, C. Le, X. Li, D. Xing, W. Zhang, Y. Wen, H. Zhang, J. Wang, Y. Yang, B. Xu, Offline pre-trained multi-agent decision transformer, Mach. Intell. Res. 20 (2) (2023) 233–248, https://doi.org/10.1007/s11633-022-1383-7
[28] Z. Han, C. Gao, J. Liu, J. Zhang, S.Q. Zhang, Parameter-efficient fine-tuning for large models: a comprehensive survey, Trans. Mach. Learn. Res. (2024) https://openreview.net/forum?id=lIsCS8b6zj.
[29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 6000–6010.
[30] J. Shang, X. Li, K. Kahatapitiya, Y.-C. Lee, M.S. Ryoo, Starformer: transformer with state-action-reward representations for robot learning, IEEE Trans. Pattern Anal. Mach. Intell. 45 (11) (2023) 12862–12877, https://doi.org/10.1109/TPAMI.2022.3204708
[31] X. Jiang, N. Gao, X. Zhang, H. Dou, Y. Deng, Value-aligned behavior cloning for offline reinforcement learning via bi-level optimization, in: International Conference

on Learning Representations (ICLR), 2025, https://openreview.net/forum?id=elTJBP7Fbv.
[32] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, Y. Wu, The surprising effectiveness of ppo in cooperative, multi-agent games, (2022). arXiv:2103.01955.
[33] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 6382–6393.
[34] P. Sunehag, G. Lever, A. Gruslys, W.M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J.Z. Leibo, K. Tuyls, T. Graepel, Value-decomposition networks for cooperative multi-agent learning based on team reward, in: Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS '18, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 2018, pp. 2085–2087.
[35] J.N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI'18/IAAI'18/EAAI'18, AAAI Press, 2018.
[36] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, S. Whiteson, QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning, in: J. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, vol. 80 of Proceedings of Machine Learning Research, PMLR, 2018, pp. 4295–4304, https://proceedings.mlr.press/v80/rashid18a.html.
[37] T. Rashid, G. Farquhar, B. Peng, S. Whiteson, Weighted qmix: expanding monotonic value function factorisation for deep multi-agent reinforcement learning, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20, Curran Associates Inc., Red Hook, NY, USA, 2020.
[38] Y. Niu, H. Zhao, L. Yu, Ma-mix: value function decomposition for cooperative multiagent reinforcement learning based on multi-head attention mechanism, in: Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems, AAMAS '24, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 2024, pp. 2402–2404.
[39] K. Son, D. Kim, W.J. Kang, D.E. Hostallero, Y. Yi, QTRAN: learning to factorize with transformation for cooperative multi-agent reinforcement learning, in: K. Chaudhuri, R. Salakhutdinov (Eds.), Proceedings of the 36th International Conference on Machine Learning, vol. 97 of Proceedings of Machine Learning Research, PMLR, 2019, pp. 5887–5896, https://proceedings.mlr.press/v97/son19a.html.
[40] Y. Yang, J. Hao, B. Liao, K. Shao, G. Chen, W. Liu, H. Tang, Qatten: A general framework for cooperative multiagent reinforcement learning, (2020). arXiv:2002.03939.
[41] J. Wang, Z. Ren, T. Liu, Y. Yu, C. Zhang, QPLEX: duplex dueling multi-agent q-learning, in: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021, OpenReview.net, 2021, https://openreview.net/forum?id=Rcmk0xxIQV.
[42] H. Li, H. He, Multiagent trust region policy optimization, IEEE Trans. Neural Netw. Learn. Syst. 35 (9) (2024) 12873–12887, https://doi.org/10.1109/TNNLS.2023.3265358
[43] M. Zhou, Z. Liu, P. Sui, Y. Li, Y.Y. Chung, Learning implicit credit assignment for cooperative multi-agent reinforcement learning, in: H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, H. Lin (Eds.), Advances in Neural Information Processing Systems, vol. 33, Curran Associates, Inc., 2020, pp. 11853–11864.
[44] T. Wang, T. Gupta, A. Mahajan, B. Peng, S. Whiteson, C. Zhang, Rode: learning roles to decompose multi-agent tasks, in: International Conference on Learning Representations, 2021, https://openreview.net/forum?id=TTUVg6vkNjK.
[45] S. Levine, A. Kumar, G. Tucker, J. Fu, Offline reinforcement learning: Tutorial, review, and perspectives on open problems, (2020). arXiv:2005.01643.
[46] T. Yamagata, A. Khalil, R. Santos-Rodriguez, Q-learning decision transformer: leveraging dynamic programming for conditional sequence modelling in offline RL, in: Proceedings of the 40th International Conference on Machine Learning, ICML'23, JMLR.org, 2023.
[47] W.-C. Tseng, T.-H. Wang, L. Yen-Chen, P. Isola, Offline multi-agent reinforcement learning with knowledge distillation, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22, Curran Associates Inc., Red Hook, NY, USA, 2022.
[48] A. Andres, L. Schäfer, S.V. Albrecht, J. Del Ser, Using offline data to speed up reinforcement learning in procedurally generated environments, Neurocomputing 618 (2025) 129079, https://doi.org/10.1016/j.neucom.2024.129079, https://www.sciencedirect.com/science/article/pii/S0925231224018502.
[49] Z. Li, W.-H. Chen, J. Yang, Y. Yan, Aid-rl: active information-directed reinforcement learning for autonomous source seeking and estimation, Neurocomputing 544 (2023) 126281, https://doi.org/10.1016/j.neucom.2023.126281, https://www.sciencedirect.com/science/article/pii/S0925231223004046.
[50] Z. Song, H. Guo, D. Jia, M. Perc, X. Li, Z. Wang, Reinforcement learning facilitates an optimal interaction intensity for cooperation, Neurocomputing 513 (2022) 104–113, https://doi.org/10.1016/j.neucom.2022.09.109, https://www.sciencedirect.com/science/article/pii/S0925231222012000.
[51] J. Jiang, Z. Lu, Offline decentralized multi-agent reinforcement learning, in: ECAI 2023 - 26th European Conference on Artificial Intelligence, September 30 - October 4, 2023 - Frontiers in Artificial Intelligence and Applications, vol. 372 of ECAI 2023, IOS Press, 2023, pp. 1148–1155, https://doi.org/10.3233/FAIA230390
[52] H. Mao, R. Zhao, H. Chen, J. Hao, Y. Chen, D. Li, J. Zhang, Z. Xiao, Transformer in transformer as backbone for deep reinforcement learning, (2023). arXiv:2212.14538.
[53] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, in: International Conference on Learning Representations (ICLR), 2016.
[54] M. Samvelyan, T. Rashid, C.S. de Witt, G. Farquhar, N. Nardelli, T.G.J. Rudner, C.-M. Hung, P.H.S. Torr, J. Foerster, S. Whiteson, The starcraft multi-agent challenge, (2019). arXiv:1902.04043.
[55] B. Ellis, J. Cook, S. Moalla, M. Samvelyan, M. Sun, A. Mahajan, J.N. Foerster, S. Whiteson, Smacv2: an improved benchmark for cooperative multi-agent reinforcement learning, in: Thirty-Seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track, vol. 36, 2023, pp. 37567–37593, https://openreview.net/forum?id=5OjLGiJW3u.
[56] E. Todorov, T. Erez, Y. Tassa, Mujoco: a Physics engine for model-based control, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2012, pp. 5026–5033 The foundational paper for the MuJoCo physics engine, upon which MAMuJoCo is built.
[57] B. Peng, T. Rashid, C.A.S. de Witt, P.-A. Kamienny, P. Torr, W. Böhmer, S. Whiteson, Facmac: factored multi-agent centralised policy gradients, in: Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 12208–12221, this paper is widely regarded as the first to propose and use the MAMuJoCo benchmark.
[58] S. Fujimoto, S.S. Gu, A minimalist approach to offline reinforcement learning, in: Proceedings of the 35th International Conference on Neural Information Processing Systems, NIPS '21, Curran Associates Inc., Red Hook, NY, USA, 2021.
[59] L. Pan, L. Huang, T. Ma, H. Xu, Plan better amid conservatism: offline multi-agent reinforcement learning with actor rectification, in: K. Chaudhuri, S. Jegelka, Song le, C. Szepesvari, G. Niu, S. Sabato (Eds.), Proceedings of the 39th International Conference on Machine Learning, vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 17221–17237, https://proceedings.mlr.press/v162/pan22a.html.

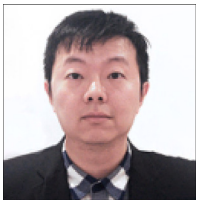
Author biography



Bohang Shi received his Bachelor's degree in Communication Engineering from Xi'an University of Posts and Telecommunications and is currently pursuing a Master's degree in Information and Communication Engineering at Xi'an Jiaotong University. His primary re-search focuses on multi-agent reinforcement learning. Representative works by Bohang Shi and his team can be found on GitHub: https://github.com/1yuan1chuan. The link to his ORCID is: https://orcid.org/0009-0009-5781-4068.



Weiqiang Jin received his M.S. in Computer Science from Shanghai University and is currently pursuing a Ph.D. in Information and Communication Engineering at Xi'an Jiaotong University, China. His main research interests include: 1. LLM-based intelligent reasoning and decision-making, as well as embodied AI; 2. AI for Science (AI4Science), particularly in drug discovery and multimodal medical information processing; 3. Reinforcement learning, with a focus on multi-agent information interaction mechanisms. He was a visiting research assistant at The University of Hong Kong from February to September 2025 and is currently a visiting scholar at The Chinese University of Hong Kong, under the supervision of Professor Irwin King. Representative works by Mr. Jin and his team can be found on GitHub: https://github.com/albert-jin. His ORCID profile is available at: https://orcid.org/0000-0002-6656-6061, and his Google Scholar page can be accessed via: https://scholar.google.com/citations?user=iDZcqAIAAAAJ.



Biao Zhao is an Associate Professor at the School of Information and Communications Engineering, Xi'an Jiaotong University. Born in 1986, he earned his B.Eng. in Information Engineering from Xi'an Jiaotong University (2005-2009) before pursuing advanced studies in Italy, where he obtained both his M.Sc. (2009-2011) and Ph.D. (2011-2015) from Politecnico di Torino. His research career spans prestigious international institutions. He served as a Postdoctoral Researcher at Politecnico di Torino (2015), contributing to computer vision algorithms and Smart City initiatives, followed by a similar role at the University of Lincoln, UK (2016-2017), where he worked on EU Horizon 2020 projects. His primary expertise lies in Reinforcement Learning for intelligent decision-making frameworks, Computer Vision (specifically object detection and 3D reconstruction), and LiDAR-based 3D navigation systems.